UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI ELECTRONICĂ

DEPARTAMENTUL DE AUTOMATICĂ ȘI ELECTRONICĂ

PROIECT DE DIPLOMĂ

CHIREA CRISTIAN MIHAIL

COORDONATOR ȘTIINȚIFIC

Asist.drd.ing. MĂMULEANU Mădălin

IULIE 2022

CRAIOVA

UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI ELECTRONICĂ

DEPARTAMENTUL DE AUTOMATICĂ ȘI ELECTRONICĂ

Dronă de colectare a datelor despre mediul înconjurător in situații de urgență

CHIREA CRISTIAN MIHAIL

COORDONATOR ȘTIINȚIFIC

Asist.drd.ing. MĂMULEANU Mădălin

IULIE 2022

CRAIOVA

DECLARAȚIE DE ORIGINALITATE

Subsemnatul CHIREA CRISTIAN-MIHAIL, student la specializarea AUTOMATICĂ ŞI INFORMATICĂ APLICATĂ din cadrul Facultății de Automatică, Calculatoare și Electronică a Universității din Craiova, certific prin prezenta că am luat la cunoștință de cele prezentate mai jos și că îmi asum, în acest context, originalitatea proiectului meu de licență:

- cu titlul *Dronă de colectare a datelor despre mediul înconjurător in situații de urgență,*
- coordonată de Asist.drd.ing. MĂMULEANU Mădălin,
- prezentată în sesiunea Iulie 2022.

La elaborarea proiectului de licență, se consideră plagiat una dintre următoarele acțiuni:

- reproducerea exactă a cuvintelor unui alt autor, dintr-o altă lucrare, în limba română sau prin traducere dintr-o altă limbă, dacă se omit ghilimele și referința precisă,
- redarea cu alte cuvinte, reformularea prin cuvinte proprii sau rezumarea ideilor din alte lucrări, dacă nu se indică sursa bibliografică,
- prezentarea unor date experimentale obținute sau a unor aplicații realizate de alți autori fără menționarea corectă a acestor surse,
- însușirea totală sau parțială a unei lucrări în care regulile de mai sus sunt respectate, dar care are alt autor.

Pentru evitarea acestor situații neplăcute se recomandă:

- plasarea între ghilimele a citatelor directe și indicarea referinței într-o listă corespunzătoare la sfârșitul lucrării,
- indicarea în text a reformulării unei idei, opinii sau teorii și corespunzător în lista de referințe a sursei originale de la care s-a făcut preluarea,
- precizarea sursei de la care s-au preluat date experimentale, descrieri tehnice, figuri, imagini, statistici, tabele et caetera,
- precizarea referințelor poate fi omisă dacă se folosesc informații sau teorii arhicunoscute, a căror paternitate este unanim cunoscută și acceptată.

Data, 24.06.2022 Semnătura candidatului,

UNIVERSITATEA DIN CRAIOVA
Facultatea de Automatică, Calculatoare și Electronică

Departamentul de Automatică și Electronică

Aprobat la data de
……
Șef de departament,
Prof. dr. ing.
Cosmin IONETE

PROIECTUL DE DIPLOMĂ

Numele și prenumele studentului/-ei:	CHIREA CRISTIAN-MIHAIL
Enunțul temei:	Dronă de colectare a datelor despre mediul înconjurător in situații de urgență
Datele de pornire:	Realizarea unui sistem controlat de la distanță ce măsoară nivelele de radiații și gaze din zonele periculoase sau neaccesibile de către personalul uman utilizând un aparat de zbor.
Conținutul proiectului:	Componente utilizate Componente Hardware Componente Software Modelul matematic a dronei Implementarea proiectului Interfațarea microcontrolerului de bord cu traductoarele Dezvoltarea aplicației desktop de control și monitorizare Dezvoltarea algoritmilor de interfațare dintre baza de date și microcontroler utilizând un SBC.
Material grafic obligatoriu:	Capturi de ecran, scheme, diagrame
Consultații:	Periodice

Conducătorul științific (titlul, nume și prenume, semnătura):	Asist.drd.ing. MĂMULEANU Mădălin
Data eliberării temei:	26.10.2021
Termenul estimat de predare a proiectului:	17.06.2022
Data predării proiectului de către student și semnătura acestuia:	24.06.2022

UNIVERSITATEA DIN CRAIOVA
Facultatea de Automatică, Calculatoare și Electronică

Departamentul de Automatică și Electronică

REFERATUL CONDUCĂTORULUI ȘTIINȚIFIC

Numele și prenumele candidatului/ei:	Chirea Cristian-Mihail
Specializarea:	Automatică și Informatică Aplicată
Titlul proiectului:	Dronă de colectare a datelor despre mediul înconjurător in situații de urgență
Locația în care s-a realizat practica de documentare (se bifează una sau mai multe din opțiunile din dreapta):	În facultate ☐ În producție ☐ În cercetare ☐ Altă locație:

În urma analizei lucrării candidatului au fost constatate următoarele:

Nivelul documentării	Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
Tipul proiectului	Cercetare ☐	Proiectare ☐	Realizare practică ☐	Altul [*se detaliază*]
Aparatul matematic utilizat	Simplu ☐	Mediu ☐	Complex ☐	Absent ☐
Utilitate	Contract de cercetare ☐	Cercetare internă ☐	Utilare ☐	Altul [*se detaliază*]

Redactarea lucrării		Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
Partea grafică, desene		Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
Realizarea practică	Contribuția autorului	Insuficientă ☐	Satisfăcătoare ☐	Mare ☐	Foarte mare ☐
	Complexitatea temei	Simplă ☐	Medie ☐	Mare ☐	Complexă ☐
	Analiza cerințelor	Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
	Arhitectura	Simplă ☐	Medie ☐	Mare ☐	Complexă ☐
	Întocmirea specificațiilor funcționale	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Implementarea	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Testarea	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Funcționarea	Da ☐	Parțială ☐	Nu ☐	
Rezultate experimentale		Experiment propriu ☐		Preluare din bibliografie ☐	
Bibliografie		Cărți	Reviste	Articole	Referințe web

Comentarii și observații	

În concluzie, se propune:

ADMITEREA PROIECTULUI	RESPINGEREA PROIECTULUI
☐	☐

Data, Semnătura conducătorului științific,

REZUMATUL PROIECTULUI

Acest proiect reprezintă un sistem dezvoltat pentru măsurarea nivelelor de substanțe periculoase gazoase dar și a elementelor radioactive prezente la locul preluării măsurătorii prin intermediul unui echipament de zbor de tip dronă. Avantajul acestei abordări stă in faptul că putem elimina nevoia prezenței umane din locurile periculoase sau foarte greu de accesat, operatorul dronei putând să o controleze prin intermediul internetului de oriunde din lume. Pentru controlul de la distanță se va folosi o aplicație software ce va trimite către o bază de date comenzile ce trebuie să fie efectuate. Aceste comenzi sunt preluate de către computerul de bord a dronei iar acesta le va furniza microcontrolerul dronei pentru a le putea efectua. Execuția măsurătorilor este un proces constant, ciclic, fiind efectuat prin intermediul traductoarelor prezente in cadrul dronei. Datele obținute sunt preluate de către computerul de bord a dronei și trimise către o bază de date, putând fi accesate de către operator prin intermediul aplicației software de control.

Întregul sistem este alcătuit din trei subsisteme. Primul îl reprezintă interfața dintre operator și sistemul de control și execuție a aparatului de zbor. El constă dintr-o interfață grafica ce permite trimiterea și accesarea de date din cadrul bazei de date. A fost implementat in limbajul de programare Java utilizând și scripturi Python. Al doilea subsistem îl reprezintă baza de date. Ea este implementată în platforma pusă la dispoziție de către Google și anume Firebase. Ultimul sistem este alcătuit dintr-un SBC (single board computer) fiind reprezentat de un RaspberryPi, un microcontroler şi anume Atmega328p şi elementele de măsurare.

Termenii cheie: Sistem Încorporat, Java, Sistem de reglare, Comunicație, Firebase

CUPRINSUL

1 Introducere....1

1.1 Scopul....1
1.2 Motivația....1

2 Componente Utilizate....2

2.1 Componente Hardware....2
2.2 Componente Software....13

3 Modelul matematic a dornei.... 18

4 Implementarea proiectului....23

4.1 Interfațarea microcontrolerului de bord cu traductoarele....23
4.2 Dezvoltarea aplicației desktop de control și monitorizare....27
4.3 Dezvoltarea algoritmilor de interfațare dintre baza de date
și microcontroler utilizând un SBC....41

4.4 Algoritmul de zbor a dronei 43

5 Concluzii....45

6 Bibliografie 46

A. Codul sursă....47

B. CD / DVD 86

LISTA FIGURILOR

FIGURA 1 DIAGRAMA FRITZING STRUCTURĂ GENERALĂ 2

FIGURA 2 SBC RASPBERRY PI 4

FIGURA 3 PLACA DE DEZVOLTARE ARDUINO 5

FIGURA 4 MODULUL MPU6050 7

FIGURA 5 DIMENSIUNILE MODULULUI MPU6050 8

FIGURA 6 MODULUL SENZOR DE GAZ MQ2 9

FIGURA 7 30A BLDC ESC 11

FIGURA 8 CIRCUIT GEIGER COUNTER 12

FIGURA 9 JAVA ŞI IDE-UL INTELLIJ 15

FIGURA 10 INTERPRETAREA CODULUI PYTHON 16

FIGURA 11 FIREBASE VERSUS ABORDARE CLASICĂ 17

FIGURA 12 STRUCTURA DONEI ŞI CADRELE INERŢIALE 18

FIGURA 13 SISTEMUL DE COORDONATE 19

FIGURA 14 MISCĂRILE DRONEI 22

FIGURA 15 CONFIGURAREA REGIŞTRILOR MPU6050 24

FIGURA 16 CALIBRAREA ACCELEROMETRULUI MPU6050 25

FIGURA 17 PRELUAREA DE MĂSURĂTORI DE LA MPU6050 25

FIGURA 18 CITIRE STARE SENZOR GAZ 26

FIGURA 19 CONFIGURAREA TIMERULUI MICROCONTROLERULUI 26

FIGURA 20 CITIRE NIVEL DE RADIAŢII IONIZANTE 27

FIGURA 21 ARHITECTURA SOFTWARE A INTERFEŢEI UTILIZATOR – FIREBASE 28

FIGURA 22 TRANSMITEREA DATELOR CĂTRE SCRIPTUL PYTHON 29

FIGURA 23 AFIŞARE INTERFAŢĂ GRAFICĂ A CONTROLULUI PAS CU PAS 30

FIGURA 24 MENIUL CONTROL SETTINGS 32

FIGURA 25 ALGORITM DE DETECŢIE ŞI CALCUL AL ÎNTRERUPERILOR 43

FIGURA 26 EXECUŢIA CICLICĂ AL ALGORITMULUI DE ZBOR 44

FIGURA 27 DRONA REALIZATĂ
45

1 INTRODUCERE

1.1 Scopul

Scopul acestui proiect este acela de a monitoriza prin mijloace autonome situația elementelor chimice gazoase și a radiațiilor ionizante din proximitatea zonelor de pericol, ce pot pune în primejdie viața oamenilor. Avansarea dronelor autonome pot determina o evoluție semnificativă în aria de dezvoltare a sistemelor de monitorizare sigură și de o mai mare acuratețe în măsurători. Pe baza datelor colectate se pot lua decizii pentru suprimarea efectelor nocive mediului înconjurător.

1.2 Motivația

Motivația pentru alegerea acestei teme a fost eliminarea factorului uman din zona compromisă. Prin intermediul utilizării aparaturii de zbor putem să folosim orice echipament de măsurare, acest lucru nu duce doar la ușurința și siguranță, dar şi la abordare multifuncțională.

2 COMPONENTE UTILIZATE

2.1 Componentele Hardware

Pentru realizarea acestui proiect s-au utilizat componentele hardware menționate mai jos:

- Raspberry Pi 4 Model B – computer pe o singură plăcuţă
- Microcontrolerul Atmega328p – placa de dezvoltare Arduino Uno
- MQ2 Gas Sensor – Senzor de gaze
- MPU6050 – Accelerometru
- 30A BLDC ESC – Controller electronic de viteză
- Tattu 2305 2450KV – Motoarele fără perii

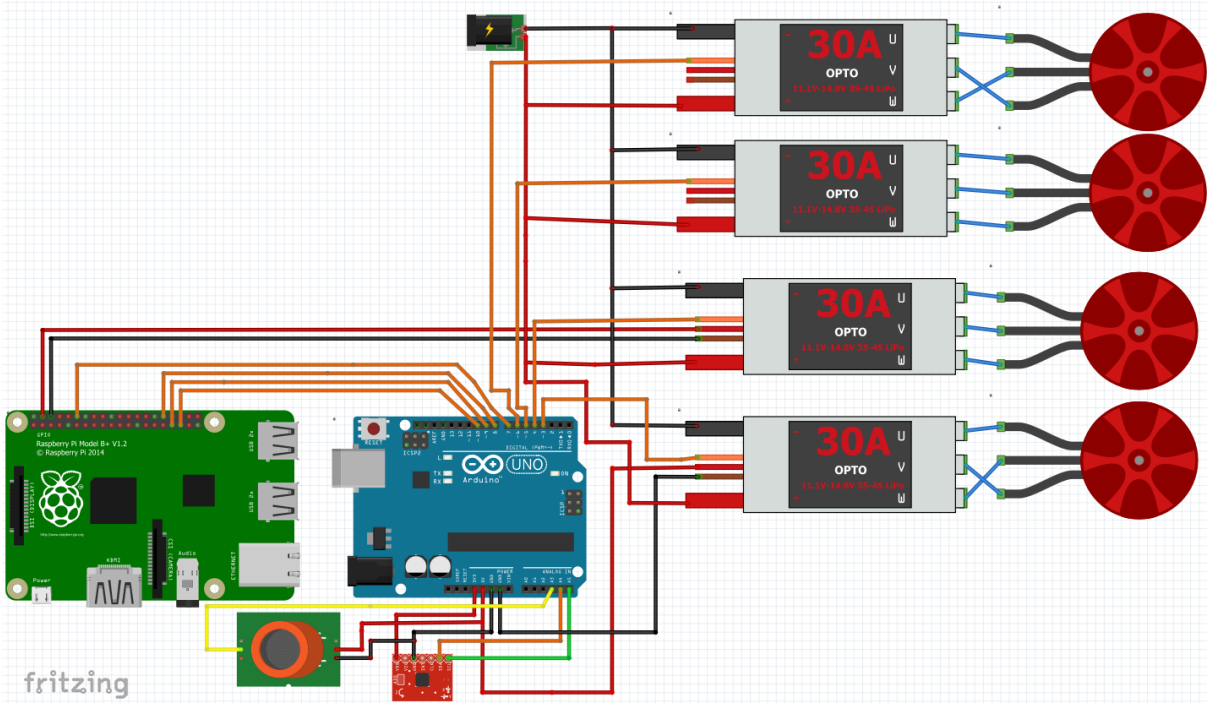


Figura 1. Diagrama Fritzing Structură Generală

Diagrama din Figura 1 a fost realizată cu Fritzing care este un mediu de simulare si proiectare hardware ce pune la dispoziţie o gama largă de componente.

Raspberry Pi 4 Model B

Raspberry PI este un computer de dimensiunile unui card de credit, mai lent ca un laptop, dar care poate oferi majoritatea aplicaţiilor acestora precum conectarea la internet, procesare de text, redare de continut video sau audio, toate la un consum redus de energie. Ce este de remarcat, faţă de un laptop sau un computer desktop, Raspberry Pi oferă utilizatorului posibilitatea de control a porturilor intrare-iesire. Astfel, prin manipulare software, se pot controla pinii de pe placă. Acesta oferă posibilitatea de a conecta diverse componente electronice specifice sistemelor înglobate: senzori, butoane, ecran LCD, relee, și crearea de noi proiecte electronice.

Raspberry Pi 4 Model B este cel mai nou model din gama de calculatoare Raspberry Pi. Comparativ cu generaţiile anterioare, viteza procesorului, performanţa multimedia, memorie şi conectivitate, a oferit creşteri semnificative de viteză şi performanţă, păstrând în același timp compatibilitatea și consumul similar de energie. Raspberry Pi 4 Model B oferă o performanță comparabilă cu sistemele de computer entry-level.

Caracteristicile cheie ale acestui produs includ un procesor quad-core pe 64 de biți de înaltă performanță, suport dual display la rezoluții de până la 4K printr-o pereche de porturi micro HDMI, decodare video hardware până la 4Kp60, până la 8 GB RAM, LAN wireless dual-band 2.4 / 5.0 GHz, Bluetooth 5.0, Gigabit Ethernet, USB 3.0, și capacitatea PoE (prin intermediul unui supliment PoE HAT separat). Conectivitatea wireless LAN pe 2 benzi si tehnologia Bluetooth au certificare modulară de conformitate, permițând proiectarea plăcii în produse finale cu o reducere semnificativă în testarea conformității, îmbunătățind atât costul, cât și timpul de introducere pe piață.

Specificatii tehnice:

- Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
- 2GB, 4GB or 8GB LPDDR4-3200 SDRAM (depending on model)
- 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless, Bluetooth 5.0, BLE
- Gigabit Ethernet
- 2 USB 3.0 ports; 2 USB 2.0 ports.
- Raspberry Pi standard 40 pin GPIO header (compatibil în totalitate cu versiunile antecedente)

- 2 × micro-HDMI ports (up to 4kp60 supported)
- 2-lane MIPI DSI display port
- 2-lane MIPI CSI camera port
- 4-pole stereo audio and composite video port
- H.265 (4kp60 decode), H264 (1080p60 decode, 1080p30 encode)
- OpenGL ES 3.1, Vulkan 1.0
- Micro-SD card slot for loading operating system and data storage
- 5V DC via USB-C connector (minim 3A*)
- 5V DC via GPIO header (minim 3A*)
- Power over Ethernet (PoE) enabled (necesita PoE HAT separat)

Microcontroller-ul Broadcom BCM2835 SoC utilizat în prima generație Raspberry Pi include un procesor ARM1176JZF-S de 700 MHz, unitatea de procesare grafică VideoCore IV (GPU) și RAM. Are o memorie cache de nivel 1 (L1) de 16 KB și o memorie cache de nivel 2 (L2) de 128 KB. Cacheul de nivel 2 este utilizat în principal de GPU. SoC este stivuit sub cipul RAM, deci numai margine sa este vizibilă. ARM1176JZ (F) -S este același procesor utilizat în iPhone-ul original, desi are o rată de ceas mai mare și este împerecheat cu un GPU mult mai rapid.

Raspberry Pi 4 folosește un SoC Broadcom BCM2711 cu un procesor ARM Cortex-A72 quad-core pe 64 de biți de 1,5 GHz, cu cache L2 partajat de 1 MB. Spre deosebire de modelele anterioare, care foloseau toate un controler de întrerupere personalizat slab potrivit pentru virtualizare, controlerul de întrerupere de pe acest SoC este compatibil cu arhitectura ARM Generic Interrupt Controller (GIC) 2.0, oferind suport hardware pentru distribuirea întreruperilor atunci când se utilizează capacitățile de virtualizare ARM.



Figura 2. SBC Raspberry PI

Microcontrolerul Atmega328p

Microcontrolerele sunt dispozitive ce conțin o unitate de procesare și alte periferice ce au rolul de a controla circuite electronice. Microcontroler-ul Atmega328p este unul dintre cele mai folosite microcontrolere, dovadă și faptul că este folosit pentru placuța de dezvoltare Arduino Uno. Este un cip microcontroler creat de către Atmel și face parte din seria de megaAVR. Acesta din urmă prezintă un procesor AVR cu o arhitectură de tip RISC (set restrâns de instrucțiuni). Caracteristicile microcontrolerului:

• Memorie program de 32kb (non-volatilă)

• Viteza de 20 MIPS (20 milioane de instrucțiuni pe secundă)

• Memorie RAM (de lucru) și ROM (non volatilă pentru stocarea de informații și după oprirea alimentării)

• Interfețe seriale de comunicare cu alte dispozitive: UART, SPI, I2C

• Numărătoare și comparatoare

Arduino UNO este o placă de dezvoltare open source realizată pe baza microcontrolerului Atmega328P.

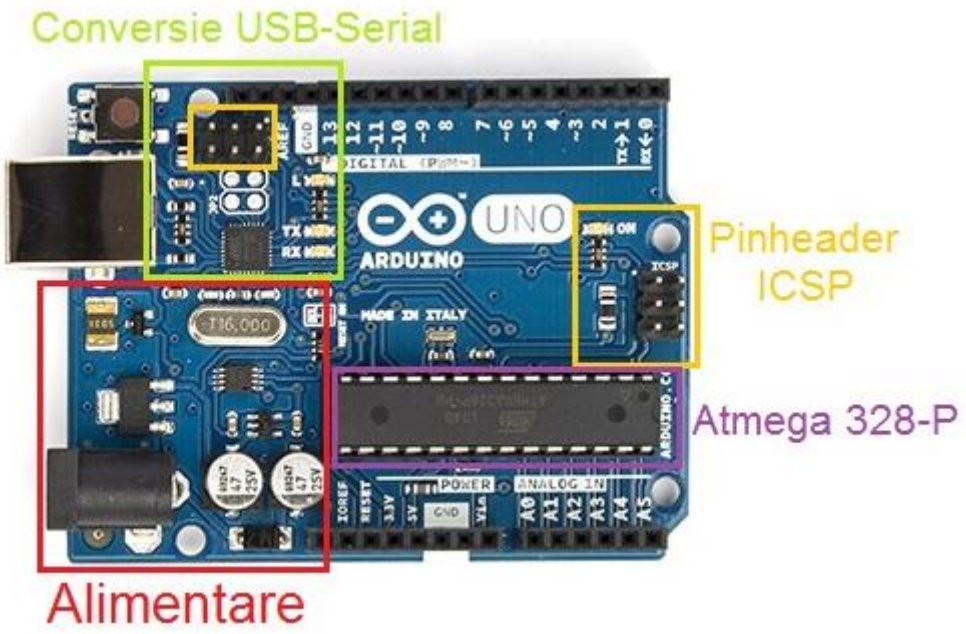


Figura 3. Placa de dezvoltare Arduino

Plăcuţa se poate alimenta la tensiuni cuprinse înntre 1.8V şi 5.5V.

Aceste microcontrolere se programează în mod inițial cu ajutorul unor dispozitive hardware adiționale numite programatoare (microcontrolerele pot fi programate inclusiv pe linia de producție) prin pinii ICSP (In-Circuit Serial Programming).

Pentru a elimina nevoia unui astfel de dispozitiv, pe aceste microcontrolere se scrie un bootloader – un mic program software ce preia instrucțiunile (codul programului) prin interfața serială și le scrie într-o zonă de memorie nevolatilă (FLASH), de unde microcontrolerul le va interpreta și rula. Astfel, microcontrolerul nostru poate interpreta acum instrucțiuni sosite pe interfață serială. Mai avem nevoie doar de o metodă de a face conversia de la USB la serial pentru a putea programa Atmega328-P prin portul USB al PC-ului. Această responsabilitate îi revine unui alt microcontroler (circuitul din chenarul verde, Figura 3). Acesta are interfață ICSP prin care poate fi reprogramat. Microcontrolerul responsabil cu conversia USB-Serial din chenarul verde poate fi înlocuit cu cipuri precum cele folosite de convertorul USB-Serial prezentat anterior.

MPU6050

Familia MPU-6000 oferă prima soluție integrată de procesare a mișcării pe 6 axe din lume care elimină dezalinierea axelor încrucișate giro/accelerare la nivel de pachet asociată cu soluțiile discrete. Dispozitivele combină un giroscop cu 3 axe și un accelerometru cu 3 axe pe aceeași matriță de siliciu împreună cu un proces digital de mișcare (DMP) integrat, capabil să proceseze algoritmi MotionFusion complexi pe 9 axe. Algoritmii MotionFusion pe 9 axe integrati ai pieselor accesează magnetometre externe sau alți senzori printr-o magistrală I2C master auxiliară, permițând dispozitivelor să adune un set complet de date ale senzorului fără intervenția procesorului sistemului. Pentru urmărirea cu precizie atât a mișcărilor rapide, cât și a mișcărilor lente, piesele au un giroscop programabil de utilizator la scară maximă de ±250, ±500, ±1000 și ±2000°/sec (dps) și un accelerometru programabil de utilizator la scară completă interval de ±2g, ±4g, ±8g și ±16g.

MPU-6050 acceptă comunicații I2C de până la 400 kHz și are un pin VLOGIC care definește nivelurile de tensiune ale interfeței. Caracteristicile suplimentare includ un senzor de temperatură încorporat și un oscilator pe cip cu o variație de ±1% în intervalul de temperatură de funcționare.

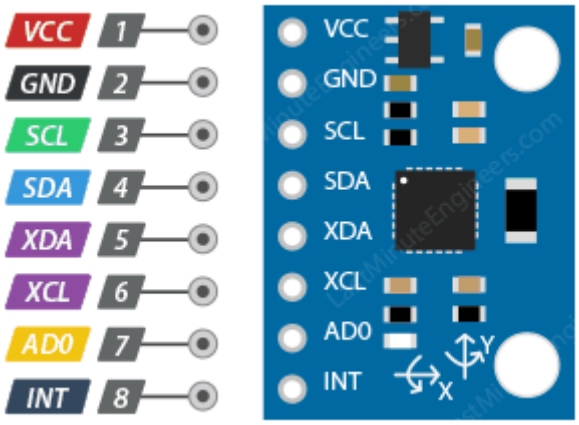


Figura 4. Modulul MPU6050

- VCC este sursa de alimentare pentru modul. Prin acest pin este furnizată o tensiune de 3.3V preluată de la plăcuţa de dezvoltare Arduino Uno
- GND este conectată la pinul de masă a placuţei Arduino
- SCL este un pin de ceas I2C. Acesta este un semnal de temporizare furnizat de dispozitivul Bus Master. Este conectat la pinul de SCL a plăcuței de dezvoltare Arduino
- SDA este un pin de date I2C. Această linie este folosită atât pentru transmitere, cât și pentru recepție. Este conectat la pinul de SDA a plăcuței de dezvoltare Arduino
- XDA este linia externă de date I2C. Busul extern I2C este pentru conectarea senzorilor externi
- XCL este linia externă de ceas I2C
- AD0 permite schimbarea adresei I2C internă a modulului MPU6050. Poate fi folosit dacă modulul este în conflict cu un alt dispozitiv I2C sau dacă sunt utilizate două MPU6050-uri pe aceeași magistrală I2C. Când pinul ADO este neconectat, adresa I2C implicită este 0x68HEX și când este conectat la o tensiune 3.3V, adresa I2C devine 0x69HEX.
- INT este ieșirea de întrerupere. MPU6050 poate fi programat pentru a ridica întreruperi la detectarea gesturilor, deplasarea, zoom-ul, derularea, detectarea atingerii și detectarea tremuratului.

Dimensiunile minimale a acestui modul permit integrarea acestuia într-o multitudine de proiecte care necesită componente de calitate ridicată dar în același timp de dimensiuni reduse.

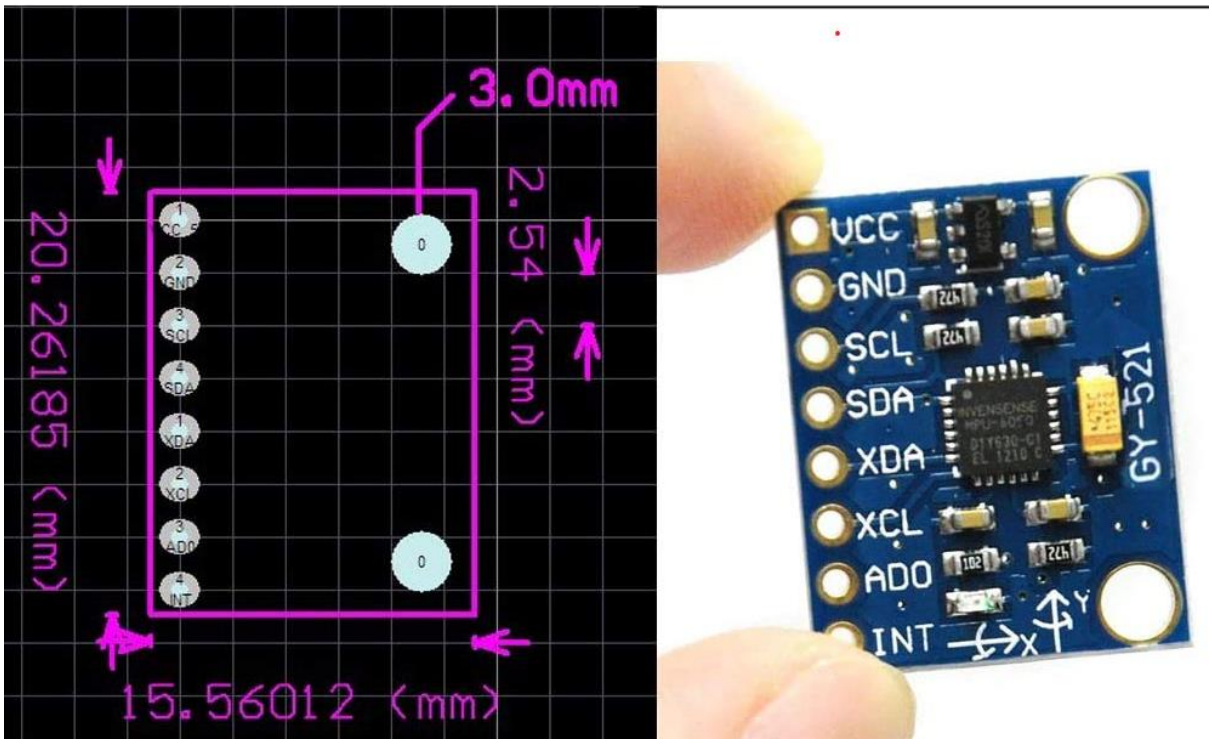


Figura 5, Dimensiunile modulului MPU6050

MQ2 Gas Sensor

Senzorul de gaz MQ2 este un senzor electronic utilizat pentru detectarea concentrației de gaze din aer, cum ar fi GPL, propan, metan, hidrogen, alcool, fum și monoxid de carbon. Senzorul de gaz MQ2 este, de asemenea, cunoscut sub numele de chemiresistor.

Conține un material de detectare a cărui rezistență se schimbă atunci când vine în contact cu gazul. Această modificare a valorii rezistenței este utilizată pentru detectarea gazului.

MQ2 este un semiconductor cu oxid de metal senzor de tip gaz. Concentrațiile de gaz din gaz se măsoară folosind un divizor de tensiune rețea prezentă în senzor. Acest senzor funcționează la tensiunea de 5V DC. Poate detecta gaze în concentrația cuprinsă între 200 și 10000ppm.

Acest senzor conține un element de detectare, în principal ceramică pe bază de oxid de aluminiu, acoperit cu dioxid de staniu, închis într-o plasă din oțel inoxidabil. Elementul de detectare are șase picioare de legătură atașate la acesta. Două cabluri sunt responsabile pentru încălzirea elementului senzitiv, celelalte patru sunt utilizate pentru semnale de ieșire. Oxigenul se adsorbe pe suprafața

materialului de detectare atunci când este încălzit în aer la temperatură ridicată. Apoi, electronii donatori prezenți în oxidul de staniu sunt atrași spre acest oxigen, prevenind astfel fluxul de curent. Când sunt prezente gaze reducătoare, acești atomi de oxigen reacționează cu gazele reducătoare, reducând astfel densitatea suprafeței oxigenului adsorbit. Acum curentul poate circula prin senzor, care a generat valori analogice de tensiune. Aceste valori de tensiune sunt măsurate pentru a cunoaște concentrația gazului. Valorile tensiunii sunt mai mari atunci când concentrația de gaz este mare.

Acest senzor este, de asemenea, utilizat pentru monitorizarea calității aerului, alarma de scurgere a gazului și pentru menținerea standardelor de mediu în spitale. În industrii, acestea sunt utilizate pentru a detecta scurgerea de gaze dăunătoare.

Alimentarea este preluată de la plăcuța de dezvoltare Arduino, care furnizează 5V DC. Valorile de gaz sunt citite de către microcontroler utilizând pinul A0 de pe modulul MQ2 şi ADC-ul microcontrolerului (pinul A3 de pe plăcuţa Arduino).



Figura 6, Modulul senzor de gaz MQ2

30A BLDC ESC

Un regulator electronic de viteză (ESC) este un circuit electronic al cărui scop principal este de a varia viteza unui motor electric și direcția acestuia.

Indiferent dacă regulatorul electronic de viteză este pentru un motor cu perii sau fără perii, el interpretează informațiile de control. Aceasta nu este ca într-o mișcare mecanică așa cum se găsește în cazul unităților servo, ci mai degrabă într-un mod care variază rata de comutare a unei rețele de tranzistori cu efect de câmp, sau mai frecvent cunoscute sub numele de FET. Comutarea rapidă a acestor tranzistori este ceea ce face ca motorul însuși să emită zgomotul său caracteristic, ascuțit, acest lucru este vizibil mai ales la viteze mai mici. Aceste FET-uri permit, de asemenea, un control mult mai fluid și mai precis al vitezei motorului și într-un mod mult mai eficient decât tipul mecanic mai vechi de regulatoare de viteză. Are o bobină rezistivă și brațe mobile, pe care le-am vedea în multe modele în urmă cu câțiva ani, mai ales în lumea modelelor de mașini. Majoritatea ESC-urilor moderne cu perie și fără perii încorporează un „Circuit de eliminare a bateriei” (BEC) pentru a regla o tensiune stabilă pentru a rula receptorul și servo-urile, acest lucru elimină necesitatea de a avea o baterie suplimentară în model. BEC-urile sunt de obicei fie „liniare”, fie „mod comutat”.

- BEC-uri liniare

Un BEC liniar folosește practic un rezistor pentru a reduce tensiunea de la 7,4v-11,1v pe care le folosim în mod obișnuit cu pachetele noastre Lipo până la 5 volți necesari pentru receptor. Cu cât este mai mare tensiunea lipo și cu cât RX consumă mai multă putere, cu atât se generează mai multă căldură pe măsură ce curentul trece peste acest rezistor. Prea mult curent se va supraîncălzi și vă va deteriora regulatorul electronic de viteză sau BEC. Ca să nu mai vorbim că irosește și multă energie electrică, mai ales cu avioanele electrice mai mari.

- BEC-uri in comutație

BEC-urile comutate de fapt pornesc și opresc tensiunea foarte rapid pentru a scădea eficient tensiunea, fără a genera căldură sau a risipi electricitate. Unele BEC-uri comutate vă permit, de asemenea, să alegeți dacă receptorul este alimentat cu 5v sau 6v, în funcție de cum și ce tip de avion zburați. Se știa că această pornire și oprire a tensiunii provoacă unele interferențe în trecut cu unele echipamente radio, dar acest tip de interferență a fost aproape eliminat în zilele noastre cu majoritatea echipamentelor moderne de 2,4 GHz. Ca regulă generală, cel mai bine este să utilizați un BEC comutat dacă bateria dumneavoastră liPo are patru sau mai multe celule și unde

sunt necesare cerințe mari de putere. ESC-ul într-un sens mai larg este controlerele Pulse Width Modulation (PWM) pentru motoare electrice. Receptorul dvs. emite un semnal PWM de 50 Hz către ESC cu o variație de la 1 ms la 2 ms, la 1 ms motorul va fi staționar, la 1,5 ms motorul va rula la jumătate de viteză, iar la 2 ms va rula la viteză maximă, desigur, veți obține un control fluid al vitezei între aceste numere. ESC-urile pentru motoarele cu perii sunt foarte diferite de ESC-urile pentru motoarele fără perii și nu sunt compatibile cu unul pe altul.Trebuie sa folosesti un ESC fara perii cu motor fara perii si un ESC cu perii cu motor cu perii.Sunt destul de usor de identificat, intrucat un ESC cu perii are 2 fire de motor si un ESC fara perii are 3 fire de motor.



Figura 7, 30A BLDC ESC

Controlul motoarelor se face utilizând 4 ESC-uri ce vor fi controlate de către microcontroler prin semnale modulate in impuls (PWM).

Tattu 2305 2450KV

Motoarele fără perii Tattu 2305 2450KV sunt cele mai bune și mai puternice motoare de drone făcute vreodată pentru drona FPV Racing. Fiecare motor quadcopter a fost atent echilibrat și testat în fabrică. Iar motorul de quadcopter Tattu este testat de mulți piloți grozavi, în timp ce infuzează avantajele tuturor celorlalte motoare din domeniul quadcopterului FPV.

Motorul quadcopter Tattu 2305 2450KV RPM este de 2450KV, curent de explozie maxim 35A, putere continuă maximă de 360W și eficiență maximă de 85%, este foarte rapid și are un răspuns rapid și este unul dintre cele mai bune motoare de curse FPV din jur. Motoarele noastre sunt potrivite în principal pentru cadru quadcopter de 180-250 mm, dimensiunile elicei compatibile sunt 5040 / 5045 elice cu două sau trei pale.

Circuitul de detecție a radiațiilor ionizante din proximitatea dronei

Pentru detecția radiațiilor trebuie folosit un circuit special care să fie conectat la sistemul nostru. Acest circuit este reprezentat in figura de mai jos.

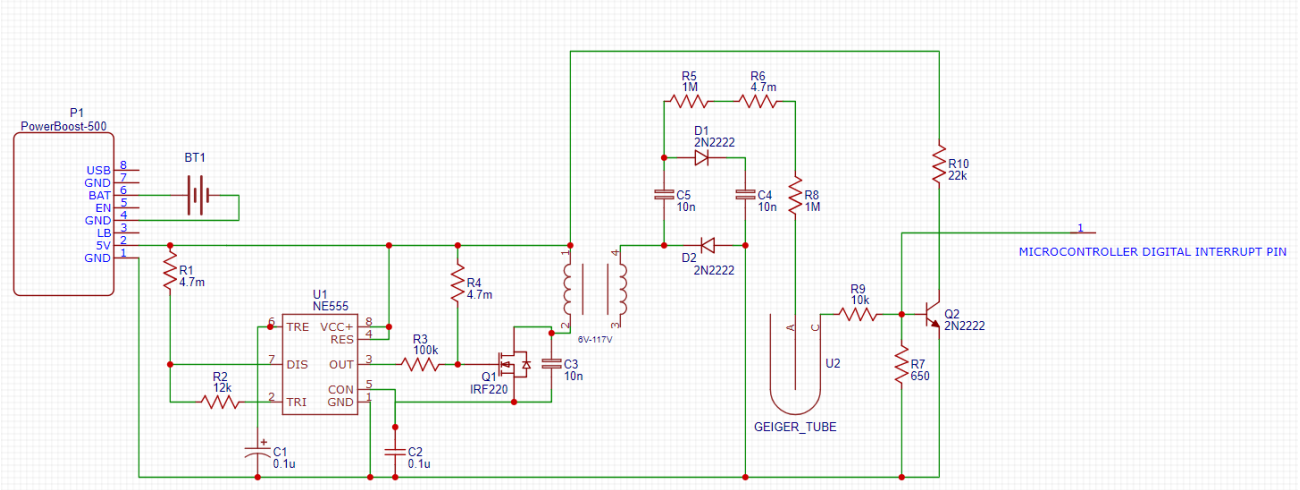


Figura 8, Circuit Geiger Counter

Circuitul furnizează la intrare 5V curent continuu. Datorita timerului NE555, putem utiliza transformatorul închizând și deschizând tranzistorul cu efect de câmp IRF220 . Astfel noi obținem o tensiune mai mare, ce va fi folosita pentru a alimenta tubul Geiger.

Când particulele ionizante străbat tubul, circuitul se închide și permite generarea semnalului de click ce este furnizat microcontrolerului.

Microcontrolerul interpretează semnalul ca pe o întrerupere și contorizează numărul de particule ionizante ce au străbătut tubul într-un minut. Astfel putem determina nivelul de radiații ionizante din proximitatea dispozitivului de zbor

2.2 Componente software

Limbajul de programare C

C este cel mai popular limbaj de programare în domeniul software pentru dezvoltarea gadgeturilor electronice. Fiecare procesor utilizat în sistemul electronic este asociat cu software-ul încorporat.

Programarea încorporată C joacă un rol cheie în îndeplinirea funcției specifice de către procesor. În viața de zi cu zi am folosit multe dispozitive electronice, cum ar fi telefonul mobil, mașina de spălat, camera digitală, etc. Toate acestea funcționează pe baza unui microcontroler care este programat de C încorporat.

În programarea sistemelor încorporate, codul C este preferat față de alt limbaj datorită următoarelor motive:

- Ușor de înțeles
- Fiabilitate ridicată
- Portabilitate
- Scalabilitate

Limbajul C nu este specific niciunui microprocesor/micro-controller sau oricărui sistem. Poate funcționa pe diverse configurații hardware. C nu necesită același set de hardware pentru a rula un program. Limbajul C este independent de platformă, în timp ce aplicațiile scrise în C nu sunt. De aici a apărut conceptul „Scrie o dată Compilează oriunde”.

Putem compila și executa un program C scris pentru un hardware într-un alt hardware cu puține modificări sau deloc. Dacă este necesară o modificare, aceasta este doar în ceea ce privește funcțiile specifice platformei (sistem de operare) și apelurile de sistem. Se spune că acesta este independent de mașină. În timp ce scrie un program în C, utilizatorul nu trebuie să cunoască detaliile specifice hardware-ului de bază ale mașinii, deoarece compilatorul se ocupă de el. Când dezvoltatorii compilează un program C pentru un anumit hardware, este generat codul de asamblare respectiv. Este apoi legat de biblioteci (statice și dinamice) pentru a da un fișier executabil. Trebuie remarcat faptul că numai codul C este independent de mașină, fișierul executabil generat este dependent de mașină, precum și de platformă (OS). Deci, ori de câte ori un cod scris în C trebuie folosit într-o altă mașină, ar trebui să fie compilat și executat din nou.

Codul C este compilat într-un executabil binar brut care poate fi încărcat direct în memorie și executat. C oferă instrucțiuni optimizate ale mașinii pentru intrarea dată, ceea ce crește performanța sistemului încorporat. Majoritatea limbajelor de nivel înalt se bazează pe biblioteci, prin urmare necesită mai multă memorie, ceea ce reprezintă o provocare majoră în sistemele încorporate.

De exemplu, în Java, avem nevoie de JVM (Java Virtual Machine) în plus față de fișierele jar (Executabile), care adaugă supraîncărcare suplimentară în ceea ce privește performanța și consumul de memorie. Limbile mai noi oferă suport pentru colectarea gunoiului, tastarea dinamică etc., ceea ce le degradează performanța.

C este un limbaj mai flexibil, mai structurat, care oferă posibilitate folosirii operatorii de biți(și, sau, not, etc). Folosind operatori pe biți putem manipula datele la nivel registrelor de memorie ceea ce este util atunci când vine vorba de sistemele încorporate.

Limbajul Java

Limbajul Java este un limbaj de programare de nivel înalt care este orientat pe obiect. El a fost creat pentru a fi portabil, cele mai multe aplicații distribuite fiind implementate in Java. A fost creat de către James Gosling in cadrul Sun Microsystems, acum fiind o filială Oracle.

El împrumuta sintaxa limbajelor de programare C si C++, dar spre deosebire de acestea el este destinat pentru o operare orientată numai pe obiect.

Un mediu integrat de dezvoltare a aplicațiilor Java este IntelliJ IDEA. El este dezvoltat de compania JetBrains si poate fi obținut in două versiuni. Una dintre ele este gratuită și anume versiunea destinată comunității, iar cealaltă este versiunea plătită destinată licențelor comerciale.

Pentru a rula Java pe dispozitivul dorit este necesar instalarea chitului de development Java și anume JDK (Java Development Kit). Acesta este pachetul de aplicații și librării care sunt necesare pentru dezvoltarea şi rularea aplicațiilor Java.

Un proiect Java conţine un set de fişiere sursă sau diferite surse cum ar fi pachete Java, poze, etc. , acestea fiind necesare rulării proiectului. Deci, el este considerat o aplicaţie unitară de sine stătătoare, fişierele şi dependinţele din componenţa proiectului fiind în strânsă legatură una faţă de cealaltă.

Fisierele Java reprezintă cea mai mica entitate din cadrul unui proiect şi în marea parte din cazuri el este conţine coduri sursă.



Figura 9, Java şi IDE-ul IntelliJ

Popularitatea limbajului Java este datorată următoarelor motive:

- **Portabilitate**: datorită platformei JVM (despre care vom povesti puțin mai jos), Java poate rula pe aproape orice tip de sistem. Java este, de asemenea, cel mai popular limbaj de Android, astfel că majoritatea aplicațiilor Android este construită în Java.

- **Versatilitate**: este disponibil pe majoritatea dispozitivelor – computere, smartphone-uri, console, centre de date etc.

- **Scalabilitate**: James Governor, unul din fondatorii RedMonk, obișnuia să spună: „Când companiile web cresc, devin shop-uri Java.” Java a fost construit astfel încât extinderea să fie făcută cu ușurință, de aceea este atât de răspândit în rândul companiilor și start-up-urilor scalabile. Spre exemplu, Twitter s-a mutat de la Ruby la Java din motive de scalare.

- **Error-freedom**: este creat astfel încât să elimine cele mai multe erori de programare – bug-uri.

- **Comunitate mare**: mulți sunt cei care utilizează și dezvoltă programe în Java, așa că asta duce la o comunitate mare de utilizatori și programatori gata să sară în ajutor oricând te confrunți cu o problemă.

- **Siguranță**: este un limbaj foarte sigur, iar programele care rulează pe internet nu prezintă riscul de a fi infectate.

- **Compatibilitate**: care garantează că aplicaţiile vechi vor continua să funcţioneze şi pe noile dispozitive.

Limbajul Python

Python este un limbaj de programare de nivel înalt, interpretat, orientat pe obiecte, cu semantică dinamică. Structurile sale de date la nivel înalt, combinate cu tastarea dinamică și legarea dinamică, îl fac foarte atractiv pentru dezvoltarea rapidă a aplicațiilor, precum și pentru utilizare ca limbaj de scripting sau lipire pentru a conecta componentele existente între ele. Sintaxa simplă, ușor de învățat a lui Python accentuează lizibilitatea și, prin urmare, reduce costurile de întreținere a programului. Python acceptă module și pachete, ceea ce încurajează modularitatea programului și reutilizarea codului. Interpretul Python și biblioteca standard extinsă sunt disponibile în formă sursă sau binară gratuit pentru toate platformele majore și pot fi distribuite gratuit

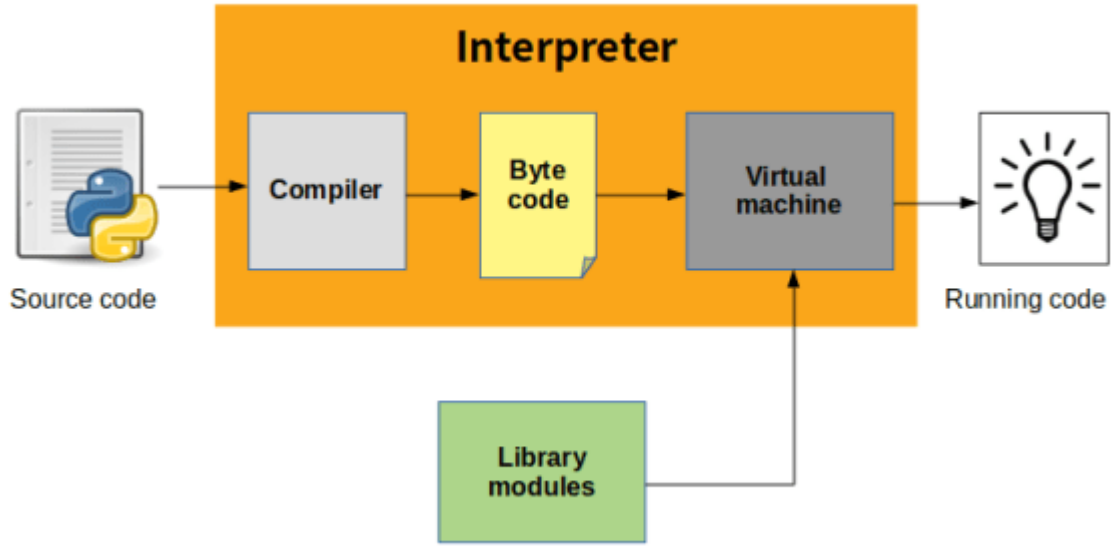


Figura 10, Interpretarea codului Python

Adesea, programatorii se îndrăgostesc de Python din cauza productivității crescute pe care o oferă. Deoarece nu există un pas de compilare, ciclul de editare-test-depanare este incredibil de rapid. Depanarea programelor Python este ușoară: o eroare sau o intrare greșită nu va provoca niciodată o eroare de segmentare. În schimb, atunci când interpretul descoperă o eroare, ridică o excepție. Când programul nu prinde excepția, interpretul imprimă o urmă de stivă. Un depanator la nivel de sursă permite inspecția variabilelor locale și globale, evaluarea expresiilor arbitrare, setarea punctelor de întrerupere, trecerea prin cod pe rând și așa mai departe. Depanatorul este scris în Python însuși,

mărturisind puterea introspectivă a lui Python. Pe de altă parte, adesea cea mai rapidă modalitate de a depana un program este să adăugați câteva instrucțiuni de tipărire la sursă: ciclul rapid de editare-test-depanare face ca această abordare simplă să fie foarte eficientă.

Firebase

Firebase este o platformă de dezvoltare web și mobilă născută în 2011 și avansată ulterior de Google în 2014. Oferă servicii pur și simplu prin implementarea câtorva linii cod, deci integrarea lui in proiecte este una uşor de făcut. Acesta poate fi integrat în site-uri web, aplicații Android sau iOS și jocuri. Oferă posibilitatea de a simplifica implementarea conexiunii, gestionarea, şi interogarea bazelor de date în cadrul programelor . În plus, există opțiunea importantă a analizei datelor pentru a reacționa în mod consecvent la comportamentul utilizatorilor, creșterea fluxului de utilizatori sau utilizarea publicității în scopuri de profit.

În acest proiect am ales utilizarea platformei deoarece simplitatea, viteza și gratuitatea acesteia sunt avantaje care fac posibilă trecerea cu vederea a timpului de răspuns a serverului în sine.

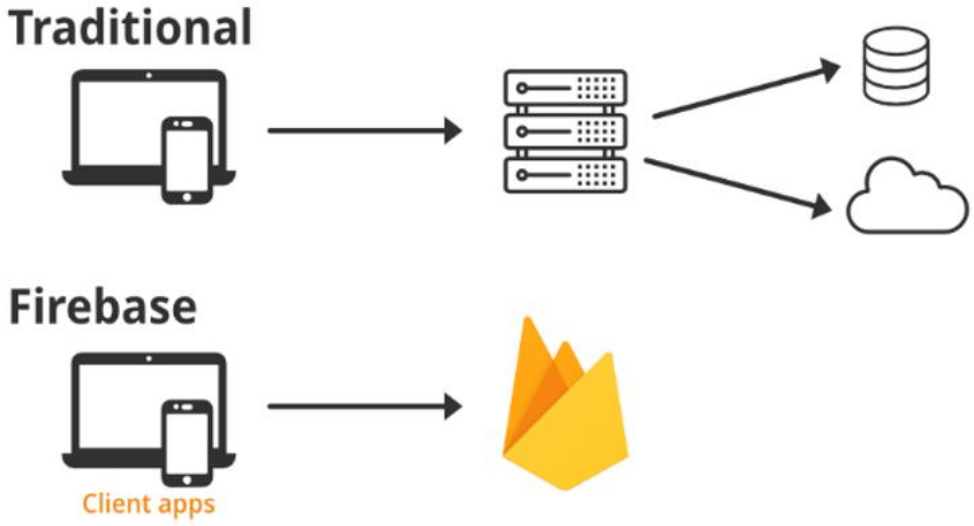


Figura 11, Firebase versus abordare clasică.

3 MODELUL MATEMATIC A DRONEI

Înainte de prezentarea ecuaţiilor diferenţiale a dronei (quadcopter) trebuie să înţelegem principiile de funcţionare ale acestuia. Quadcopterul mai este cunoscut şi sub numele de quadrotor şi reprezintă un elicopter, de obicei controlat de la distanţă, care are patru rotoare. Rotoarele sunt aşezate echidistant unul faţă decentrul de greutate al dronei, îndreptate deasupra aparatului, şi rotaţia acestora esdte controlată de către microcontrolerul de bord. Microcontrolerul realizeaza acest control prin modificarea vitezelor unghiulare ale rotoarelor care sunt rotite de către motoarele electrice.

Drona de tip quadcopter este un design tipic vehiculelor aeriene de supraveghere, căutare, salvare, inspecţii de construcţii şi acum analiza datelor critice ce pot pune integritatea oamenilor în pericol, fără pilot, datorită simplităţii structurale.

Structura dronei este prezentată în figura de mai jos.

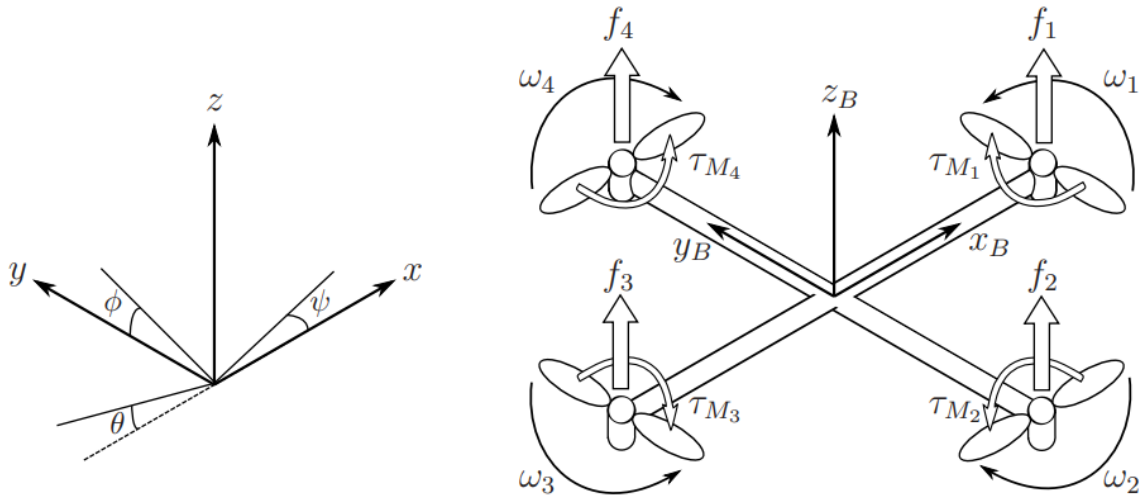


Figura 12, Structura donei şi cadrele inerţiale

După cum putem să constatăm, rotaţia rotoarelor este una opusa una faţă de cealaltă. Două dintre rotoare au acelaşi sens de rotaţie iar cele două rămase sens de rotaţie opus cu primele.

Acest lucru nu este unul aleator. Această configuraţie este utilizată de producătorii de drone pentru a putea combate momentul inertial al vitezei unchiulare ce ar determina drona să se rotească in jurul axei propri în direcţia de rotaţie a rotoarelor. Astfel, în această configuraţie, cu cele două seturi de rotoare care se rotesc în sirecţii opuse, momentul unghiular total este zero.

Quadcopterul are 6 grade de libertate. Aceasta înseamnă că sunt necesare 6 variabile pentru a-și exprima poziția și orientarea în spațiu (x, y, z, φ, θ și ψ). Variabilele x, y și z reprezintă distanțele centrului de masă al quadcopterului de-a lungul axelor x, y și respectiv z față de cadrul de referință fix. Din aceste 6 grade de libertate noi putem controla doar unghiurile φ, θ și ψ pentru a ne deplasa în spaţiu

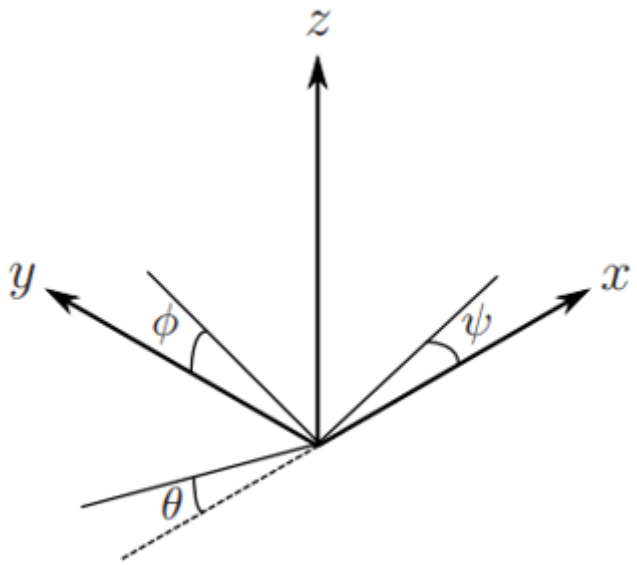


Figura 13, Sistemul de coordonate

Poziția liniară absolută a quadcopterului este definită în cadrul inerțial **x**,**y**,**z** cu **ξ**. Atitudinea, adică poziția unghiulară, este definită în cadrul inerțial cu trei unghiuri Euler **η**. Unghiul de înclinare **θ** determină rotația quadcopterului în jurul axei **y**. Unghiul de rulare **φ** determină rotația în jurul axei **x** și viarea unghiul **ψ** în jurul axei **z**. Vectorul **q** conține vectorii de poziție liniară și unghiulară

$$\xi = \begin{bmatrix} x \\ y \\ z \end{bmatrix}; \quad \eta = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}; \quad q = \begin{bmatrix} -\xi \\ \eta \end{bmatrix}$$

Punctul de origine se află in centrul de greutate al aparatului de zbor. În cadrul corpului dronei vitezele liniare sunt determinate de $\boldsymbol{V_B}$ și vitezele unghiulare de $\boldsymbol{\nu}$

$$V_B = \begin{bmatrix} v_{x,B} \\ v_{y,B} \\ v_{z,B} \end{bmatrix}, \nu = \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

Matricea de rotaţie a corpului dronei în cadru inerţial este

$$R = \begin{bmatrix} C_\psi C_\theta & C_\psi S_\theta S_\phi - S_\psi C_\phi & C_\psi S_\theta C_\phi + S_\psi S_\phi \\ S_\psi C_\theta & S_\psi S_\theta S_\phi + C_\psi C_\phi & S_\psi S_\theta C_\phi - C_\psi S_\phi \\ -S_\theta & C_\theta S_\phi & C_\theta C_\phi \end{bmatrix}$$

unde $\boldsymbol{S_x} = \mathbf{sin}\,(\boldsymbol{x})$ şi $\boldsymbol{C_x} = \mathbf{cos}\,(\boldsymbol{x})$. Matricea $\boldsymbol{R}$ de rotaţie este ortogonală deci inversa acesteia fiind egală cu transpusa acesteia.

Matricea de transformare a vitezelor unghiulare din cadrul inerţial la cadrul de referinţă a corpului este notat cu $\boldsymbol{W_\eta}$, iar transformarea din cadrul de referinţă a corpului la cadrul inerţial reprezintă inversa matricii $\boldsymbol{W_\eta}$ notată cu $\boldsymbol{W_\eta^{-1}}$.

$$\dot{\eta} = W_\eta^{-1}\nu, \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & S_\phi T_\theta & C_\phi T_\theta \\ 0 & C_\phi & -S_\phi \\ 0 & S_\phi / C_\theta & C_\phi / C_\theta \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix},$$

$$\nu = W_\eta \dot{\eta}, \begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} 1 & 0 & -S_\theta \\ 0 & C_\phi & C_\theta S_\phi \\ 0 & -S_\phi & C_\theta C_\phi \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix},$$

unde $\boldsymbol{T_x = \tan(x)}$. Matricea $\boldsymbol{W_\eta}$ este inversabilă daca şi numai dacă se respectă condiţia că

$$\theta \neq (2k-1)\phi/2, (k \in \mathbb{Z}).$$

Prin definiţie structura dronei trebuie să fie una simetrică, cu cele patru braţe ce susţin motoarele aliniate pe axele de referinţă ale corpului x şi y. Astfel, matricea de inerţie este reprezentată de matricea diagonală $\mathbf{I}$ unde $\mathbf{I_{xx}} = \mathbf{I_{yy}}$.

$$\boldsymbol{I} = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix}$$

Viteza unghiulară a rotorului i, notată cu ω_i (Figura 12), genereaza o forţă f_i care are aceeaşi direcţie cu axa rotorului. Totodată viteza unghiulară a rotorului şi implicit acceleraţia acestuia va crea un cuplu notat cu τ_{Mi} în jurul axei rotorului.

$$f_i = k\omega_i^2, \tau_{M_i} = b\omega_i^2 + I_M\dot{\omega}_i,$$

unde k reprezintă constanta de ridicare , constanta de frânare este reprezentată prin variabila b iar momentul inerţial al rotorului este I_M. De obicei efectul eccelerației unghiulare notate cu $\dot{\omega}_i$ este atât de mic încât el se poate neglija, deci îl putem omite.

Toate forţele combinate aplicate rotorului dau naştere deplasării peverticală, ridicarea pe axa z. Cuplul τ_B este compus din cuplurile corespunzătoare unghiurilor cadrului corpului $\tau\varphi$, $\tau\theta$ şi $\tau\psi$.

$$T = \sum_{i=1}^{4} f_i = k \sum_{i=1}^{4} \omega_i^2, \boldsymbol{T}^B = \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix},$$

$$\boldsymbol{\tau}_B = \begin{bmatrix} \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} lk(-\omega_2^2 + \omega_4^2) \\ lk(-\omega_1^2 + \omega_3^2) \\ 4 \\ \sum_{i=1}^{4} \tau_{M_i} \end{bmatrix},$$

unde *l* reprezintă distanţa dintre rotor şi centrul de greutate a dronei.

Astfel, mișcarea de rotaţie in jurul coordonatei x este dobândită prin scăderea vitezei a 2-a a rotorului și creșterea vitezei a 4-a a rotorului. În mod similar, mișcarea de înclinare este dobândită de scăderea vitezei rotorului 1 și mărirea vitezei rotorului al 3-lea. Mișcarea de rotire este dobândită prin creșterea vitezelor unghiulare a două rotoare opuse și scăzând vitezele celorlalte două.

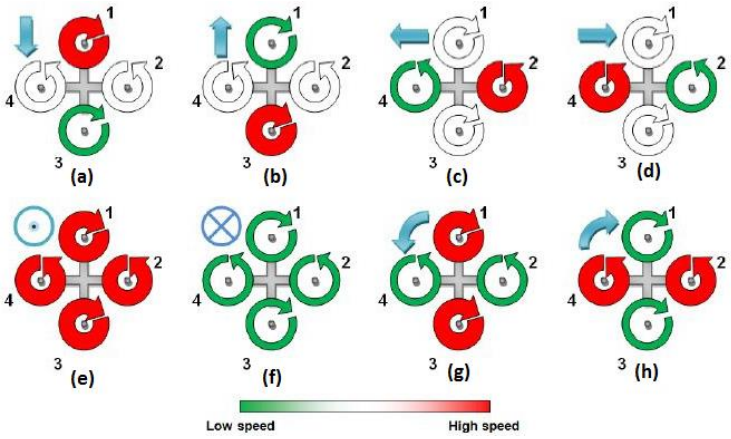


Figura 14, Miscările dronei

4 IMPLEMENTAREA PROIECTULUI

4.1 Interfațarea microcontrolerului de bord cu traductoarele

Citirea datelor senzorilor de bord a dronei este realizată prin interfețe software şi hardware. Din punct de vedere hardware, senzorii propriu ziși (MQ2 şi MPU6050, neluând in seama tubul Geiger) se află integrați pe plăcuțe de dezvoltare ce fac mai ușor conectivitatea acestora cu orice tip de echipament logic de calcul. Astfel, se conectează prin intermediul pinilor la microcontrolerul Atmega328p astfel

- Senzorul MQ2 de gaz este conectat la pinul ADC3 al microcontrolerului. Pinul ADC3 unul dintre cei șase pini ce sunt utilizați în măsurarea sau generarea semnalelor analogice cuprinse între valorile de 0V şi 5V. Acest fapt este posibil cu ajutorul celor șase circuite de conversie analog-digitale numite ADC (Analog to Digital Converter). Acestea au o rezoluție de 10 biți. În acest caz, putem măsura variația de tensiune cauzată de detecția gazelor din atmosferă cu ajutorul senzorului.
- Senzorul MPU6050 transmite date către microcontroler utilizând protocolul de comunicaţie I2C. Acesta are conectați pinii SCL şi SDA la pinii corespunzători SCL şi SDA ai microcontrolerului Atmega328p. Acest protocol de comunicaţie se bazează pe un tip de magistrală master-slave. Dispozitivul master va furniza tensiune pe pinul SCL(între 3.3V şi 5V în cazul de faţă) si pune SDA în 0V, astfel toate dispozitivele slave sunt pregatite pentru a receptiona date. Daca doua sau mai multe dispozitive master doresc sa-si asume bus-ul la un moment dat, primul dispozitiv care va furniza tensiune pe pinul de SCL, va prelua controlul bus-ului. Datele pot incepe a fi trasmise. Dispozitivul master va continua sa genereze impulsuri de ceas la un interval regulat, iar datele vor fi plasate pe SDA, fie de dispozitivul master fie de cel slave, in functie de bitii Read/Write transmisi in cadrul anterior. Numarul de cadre de date este arbitar.

Codul de interfaţare cu accelerometrul MPU6050 începe cu configurarea registrilor din memoria MPU6050. Această configurare ne este prezentată in cartea tehnică a senzorului.

```
void setupMpu6050Registers() {
    // Configuratie power management
    Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
    Wire.write(0x6B);                    // Request PWR_MGMT_1 de la registru
    Wire.write(0x00);                    // Aplica configuratia la registru
    Wire.endTransmission();              // Stop

    // Configura gyro sensitivitate
    Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
    Wire.write(0x1B);                    // Request GYRO_CONFIG de la registru
    Wire.write(0x08);                    // Aplica configuratia la registru : ±500°/s
    Wire.endTransmission();              // Stop

    // Configuratia accelerometrului
    Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
    Wire.write(0x1C);                    //Request ACCEL_CONFIG de la registru
    Wire.write(0x10);                    // Aplica configuratia la registru : ±500°/s
    Wire.endTransmission();              //Stop

    // Configuratie filtru digital trece jos
    Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
    Wire.write(0x1A);                    // Request CONFIG de la registru
    Wire.write(0x03);                    // Seteaza filtru digital trece jos la ~43Hz
    Wire.endTransmission();              // Stop
}
```

Figura 15, Configurarea regiştrilor MPU6050

Se încarcă adresele fiecărui registru cu datele reprezentate în hexazecimal din codul de mai sus.

Ca şi în cazul setării registrilor, acest tip de calcul algoritmic ne este prezentat in cartea tehnică a senzotului. Astfel, calibrarea senzorului este fâcută prin preluarea de măsurători timp de 2000 de ciclii.

```
void calibrateMpu6050() {
    int max_samples = 2000;

    for (int i = 0; i < max_samples; i++) {
        readSensor();

        gyro_offset[X] += gyro_raw[X];
        gyro_offset[Y] += gyro_raw[Y];
        gyro_offset[Z] += gyro_raw[Z];

        //Genereaza impuls de comandă către ESC-uri pentru a le arma
        PORTD |= B11110000;       // Seteaza pin #4 #5 #6 #7 pe HIGH
        delayMicroseconds(1000); // Stai 1000µs
        PORTD &= B00001111;       // Masca ce pune pinii pe LOW

        // asteapta 3 ms pentru urmatorul loop
        delay(3);
    }

    // Calcul compensare medie
    gyro_offset[X] /= max_samples;
    gyro_offset[Y] /= max_samples;
    gyro_offset[Z] /= max_samples;
}
```

Figura 16, Calibrarea accelerometrului MPU6050.

Funcţia de readSensor preia măsurătorile.

```
void readSensor() {
    Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
    Wire.write(0x3B);                    // Trimitere request de pornire
    Wire.endTransmission();              // Stop transmisiei
    Wire.requestFrom(MPU_ADDRESS,14);    // Citeste 14 bytes de la MPU

    // Asteapta pana când s-ai citit toti bitii
    while(Wire.available() < 14);

    // Adaugă bitii in containarele acc_raw[X],acc_raw[Y],acc_raw[Z],temperatura,gyro_raw[X],gyro_raw[Y],gyro_raw[Z]
    acc_raw[X]  = Wire.read() << 8 | Wire.read();
    acc_raw[Y]  = Wire.read() << 8 | Wire.read();
    acc_raw[Z]  = Wire.read() << 8 | Wire.read();
    temperatura = Wire.read() << 8 | Wire.read();
    gyro_raw[X] = Wire.read() << 8 | Wire.read();
    gyro_raw[Y] = Wire.read() << 8 | Wire.read();
    gyro_raw[Z] = Wire.read() << 8 | Wire.read();
    // Acestia conţin acum datele neprelucrate(bitii) preluate de la senzor
}
```

Figura 17, Preluarea de măsurători de la MPU6050

Acum avem acces la datele furnizate de către accelerometru. Pentru ca să citim senozorul de gaz trebuie doar să citim valoarea furnizată de ADC3

```
void citireSenzorGaz()
{

    #if(GAS_SENSOR_PRESENT)
      return analogRead(A3);
    #elif
      return ;
    #endif

}
```

Figura 18, Citire stare senzor gaz.

unde GAS_SENSOR_PRESENT este un define configurabil ce permite programatorului să ia decizia de a înlatura senzorul în totalitate din componenta software, astfel făcând codul fiabil şi transferabil către alte echipamente care au componente hardware diferite.

Pentru detecţia de particule ionizante ce trec prin tub, este nevoie să avem configurat un timer pentru a detecta într-o secundă câte particule au străpuns tubul Geiger. In partea de setup a programului trebuie încărcaţi registrii timerului pentru a seta prescalorul şi a activa timerul.

```
void setup() {

    #if(GEYGER_SENSOR_PRESENT)
      // Start timer geyger
        noInterrupts();           // disable all interrupts

        pinMode(interruptPinTemp, INPUT_PULLUP);
        pinMode(interruptPinSlanost, INPUT_PULLUP);


        TCCR1A = 0;
        TCCR1B = 0;
        timer1_counter = 31250; //34286;   // preload timer 65536-16MHz/256/2Hz

        TCNT1 = timer1_counter;   // preload timer
        TCCR1B |= (1 << CS12);    // 256 prescaler
        TIMSK1 |= (1 << TOIE1);   // enable timer overflow interrupt

        interrupts();             // enable all interrupts
#endif
//endif GEYGER_SENSOR_PRESENT
```

Figura 19, Configurarea timerului microcontrolerului.

După care, dacă pe pinul digital doisprezece a microcontrolerului este detectată o întrerupere, contorizăm intr-o variabilă statică acel click. Dupa o secundă transmitem acea variabilă pe portul serial a microcontrolerului şi resetam variabila statică cât şi timerul.

```
#if(GEYGER_SENSOR_PRESENT)

ISR(PCINT0_vect) {

    // Geiger Click
        if (PINB & B00010000) {
           ticks = ticks + 1;
        }

}

#endif
//endif GEYGER_SENSOR_PRESENT


#if(GEYGER_SENSOR_PRESENT)
     ISR(TIMER1_OVF_vect)        // intrerupere generata de timer la interval de o secunda
      {
            Serial.Write((int)ticks);
            ticks = 0;
            TCNT1 = timer1_counter;   // reseteaza timer
      }
#endif
//endif GEYGER_SENSOR_PRESENT
```

Figura 20, Citire nivel de radiaţii ionizante.

La fel, programatorul poate decide dacă sistemul lui are nevoie de această funcţionalitate setând definiţia GEYGER_SENSOR_PRESENT.

4.2 Dezvoltarea aplicației desktop de control și monitorizare

Pentru ca aparatul de zbor (dronă) să poată fi pilotat de oriunde din lume avem nevoie de o aplicaţie care să permită conexiunea la dronă prin intermediul internetului. Această conexiune se poate realiza printr-o bază de date, datele reprezentând comenzi şi măsurători înspre şi dinspre dronă.

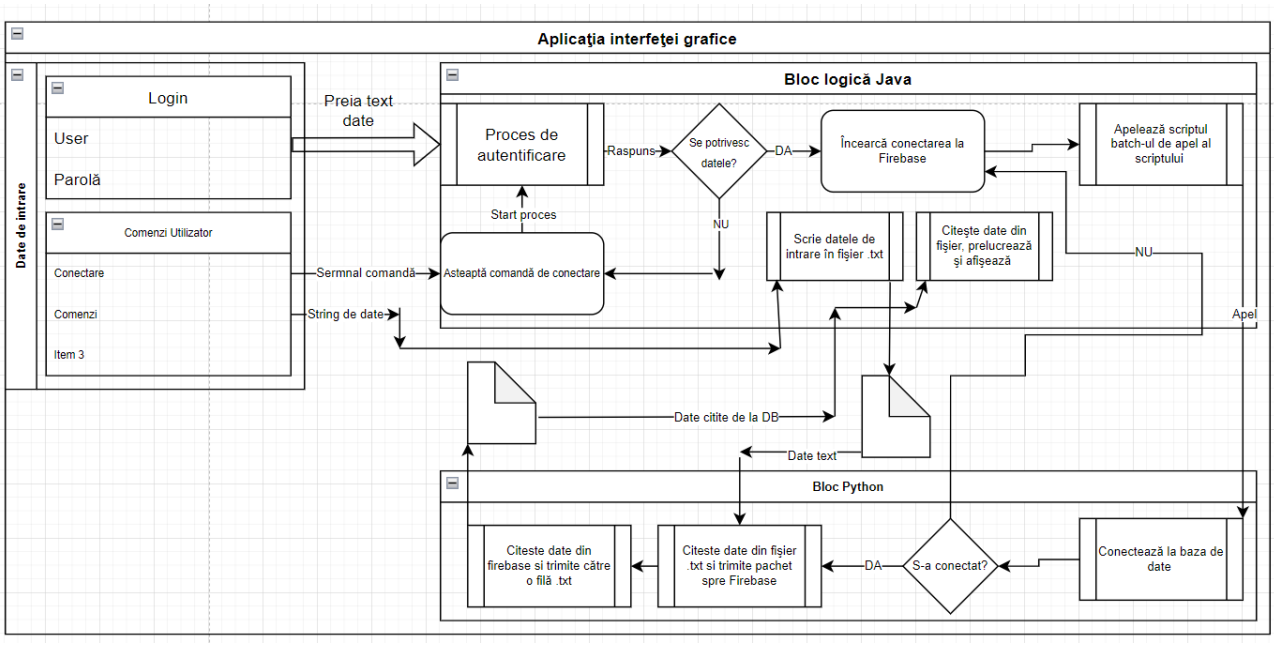


Figura 21, Arhitectura software a interfeței utilizator – Firebase

Fiind un aparat destul de complex, drona trebuie să fie securizată, accesul neautorizat să fie imposibil. Deci avem nevoie de o metodă prin care numai persoanele autorizate să poată accesa. Această metodă este una de tip autentificare prin username şi parolă. Dacă aceste date au fost furnizate corect atunci se poate continua la procesul de conectare cu Firebase. Dacă nu atunci se reia autentificarea utilizatorului.

Conectarea cu Firebase sem face prin intermediul unui script Python. Interfaţa grafică este scrisă in limbajul de programare Java, deci vom rula un batch file ce va permite utilizarea scriptului Python. Dacă procesul de conectare nu a fost autorizat atunci se reâncearcă reconectarea.

Daca este autorizat atunci începe transmisia de date către Firebase. Interfaţa grafică preia datele în format numeric, le rescrie într-o filă cu extensie txt, iar scriptul de Python preia textul din această filă şi o prelucrează. Datele prelucrate sunt trimise către baza de date in format text.

Codul de mai jos este codul Java ce scrie datele in acest fişier text şi apelează scriptul batch.

```
try{

    FileWriter myWritter = new FileWriter( fileName: "D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");

    myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
    myWritter.write( str: "|");
    myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
    myWritter.write( str: "|");
    myWritter.write(String.valueOf(YAWSPINNER.getValue()));
    myWritter.write( str: "|");
    myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));

    myWritter.close();

    process = new ProcessBuilder( ...command: "D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
    //should integrate when send button is triggered
    quadcopterLogTextArea.setText("Sending to client:\n\n" +
            "Pitch value: " + PITCHSPINNER.getValue()  +"\n"+
            "Roll value: "  + ROLLSPINNER.getValue()   +"\n"+
            "Yaw value: "   + YAWSPINNER.getValue()    +"\n" +
            "Throttle value" + THROTTLESPINNER.getValue() + "\n");


}catch(Exception except) {
    System.out.println("Exception Raised" + except.toString());
}
```

Figura 22, Transmiterea datelor către scriptul Python

Scriptul batch ce face apel către scriptul de Python este cel apelat de către **proccess**. Conţinutul acestuia este

@echo off

"C:\Users\User\AppData\Local\Programs\Python\Python310\python.exe"
"D:\projects\AppConection\src\com\codebind\sender.py"

pause

unde prima directivă reprexintă locaţia interpretatorului de python. A doua directiva reprezintă scriptul de Python ce trebuie rulat pentru a trimite datele către Firebase. Conţinutul scriptului de

Python este următorul :

```
from firebase import firebase
import time
#read from log file
f = open("D:\projects\AppConection\src\com\codebind\log.txt","r")
#data input
data_raw = f.read()
data_input = {"data" : data_raw}
```

```
print(f.read())
a = time.time_ns()
print(a)
#connection to firebase database
firebase = firebase.FirebaseApplication("https://database-licenta-default-rtdb.europe-west1.firebasedatabase.app/", None)
#delete data from DB to create room for the next command
delete = firebase.delete('/fir-database-3fb8a-default-rtdb', 'Customer')
#put data inside database
   #this represents values stored for the reciever RPY
data = data_input
   #post data inside firebase rep
post = firebase.post('/fir-database-3fb8a-default-rtdb/Customer', data)
#print(post)
```

Acesta realizează conexiunea cu baza de date care are un url unic. Procesează datele, le trimite în format text. Dar înainte de a le trimite, şterge datele stocate pentru a folosi cât mai puţin spaţiu.

Toate prezentate mai sus reprezintă configurația clasică a funcționalității programului.Interfaţa grafică are urmatoarea afişare:

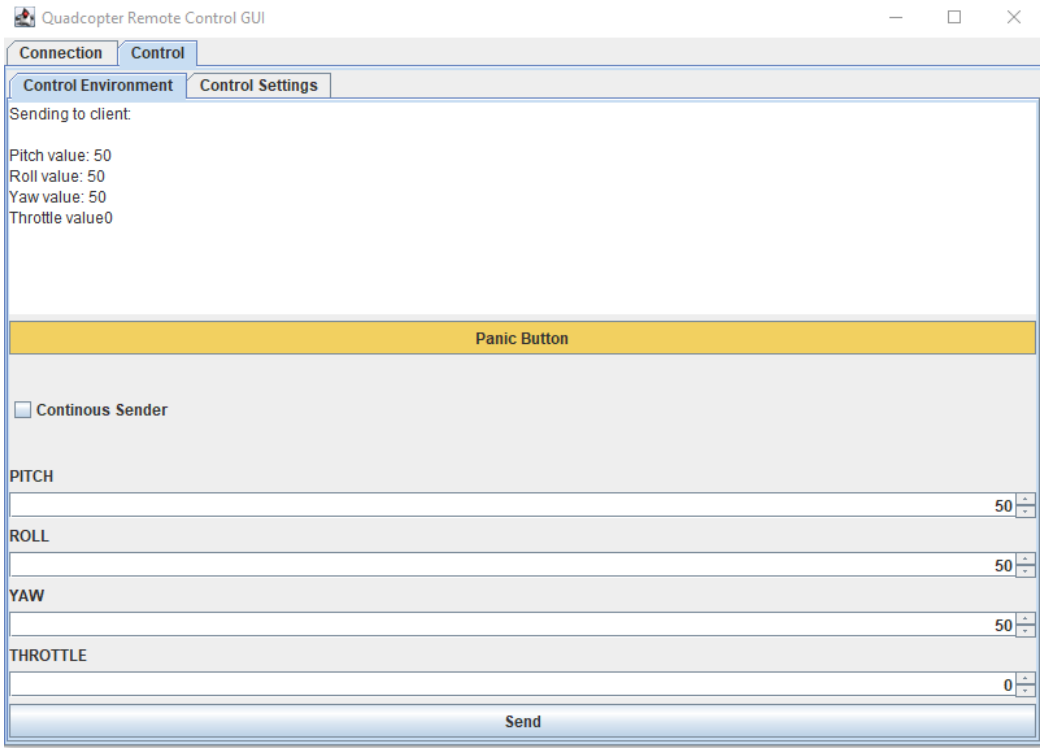


Figura 23, Afişare interfaţă grafică a controlului pas cu pas

Dupa logarea si autentificarea utilizatorului, din meniul superior putem alege fereastra de control. În această fereastră putem alege două modalităţi de control. Una este pas cu pas, la fiecare apăsare a butonului de Send se va trimite comanda către dronă. A doua este când butonul de Continous Sender este activ, atunci se trimit încontinuu comenzile către dronă.

Funcționalitatea este descrisă în codul prezentat mai sus, dar pentru partea de transmisie continuă este implementat acest algoritm

```
Timer timer = new Timer(200,new ActionListener(){
   public void actionPerformed(ActionEvent e)
   {
         if(standaloneConfigurationRadioButton.isSelected())
         {
            if(connectionRequired == 1 && continousSenderCheckBox.isSelected())
            {
               quadcopterLogTextArea.setText("Sending to client:\n\n" +
                     "Pitch value: " + PITCHSPINNER.getValue()  +"\n"+
                     "Roll value: "  + ROLLSPINNER.getValue()   +"\n"+
                     "Yaw value: "   + YAWSPINNER.getValue()    +"\n" +
                     "Throttle value" + THROTTLESPINNER.getValue() + "\n";


               try{
                  FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
                  myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
                  myWritter.write("|");
                  myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
                  myWritter.write("|");
                  myWritter.write(String.valueOf(YAWSPINNER.getValue()));
                  myWritter.write("|");
                  myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
                  myWritter.close();
                  process = new ProcessBuilder("\\src\\com\\codebind\\serverSender.bat").start();
                  //should integrate when send button is triggered
                  quadcopterLogTextArea.setText("Sending to client:\n\n" +
                        "Pitch value: " + PITCHSPINNER.getValue()  +"\n"+
                        "Roll value: "  + ROLLSPINNER.getValue()   +"\n"+
```

```
                    "Yaw value: "   + YAWSPINNER.getValue()    +"\n" +
                    "Throttle value" + THROTTLESPINNER.getValue() + "\n");

            }catch(Exception except) {
               System.out.println("Exception Raised" +except.toString());
            }
         }
      }
   }
});
timer.start();
```

Acesta reprezintă o întrerupere a timerului programului. La interval de 200 de milisecunde se testează dacă butonul de continous reading este selectat. Dacă da atunci trimite datele exact ca în modul simplu, dar că le trimite ciclic la intervale de 200 de milisecunde. Drona se mai poate controla şi din tastatură. Modul de control din tastatură este selectat când intrăm în meniul de Control Settings.

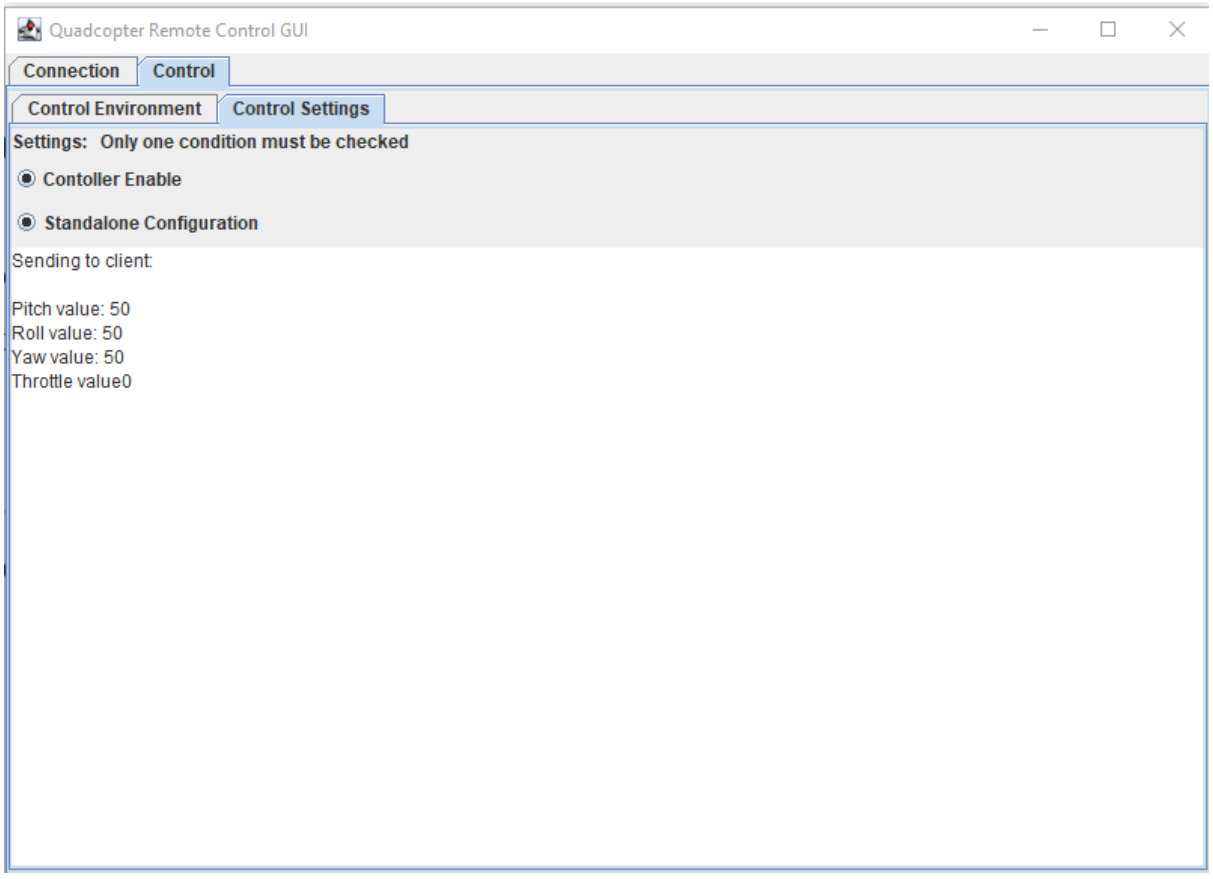


Figura 24, Meniul Control Settings

După cum putem vedea avem inclus un buton de Controller Enable. Acest control permite utilizatorului să trimită comenzi prin intermediul tastaturii, logica din spate fiind cea de la jocurile video.

```
//CONTROLLER ENABLED
contollerEnableRadioButton.addKeyListener(new KeyAdapter() {
    @Override public void keyPressed(KeyEvent e) {
        int keyCode = e.getKeyCode();
        if (keyCode == KeyEvent.VK_UP && connectionRequired == 1) {
            System.out.println("Up Arrrow-Key is pressed!");
            try{
                int a = Integer.valueOf(String.valueOf(THROTTLESPINNER.getValue()));
                if(a >= 100 )
                {
                    a = 100;
                }
                else
                {
                    THROTTLESPINNER.setValue(++a);
                }
                FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
                myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
                myWritter.write("|");
                myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
                myWritter.write("|");
                myWritter.write(String.valueOf(YAWSPINNER.getValue()));
                myWritter.write("|");
                myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
                myWritter.close();
                process = new ProcessBuilder("\\src\\com\\codebind\\serverSender.bat").start();
                AutomaticControlLog.setText("Sending to client:\n\n" +
                        "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                        "Roll value: " + ROLLSPINNER.getValue() + "\n" +
                        "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                        "Throttle value" + THROTTLESPINNER.getValue() + "\n");
            }catch(Exception except) {
```

```
            System.out.println("Exception Raised" + except.toString());
        }
    }
    else if (keyCode == KeyEvent.VK_DOWN) {
        System.out.println("Down Arrrow-Key is pressed!");
        try{
            int a = Integer.valueOf(String.valueOf(THROTTLESPINNER.getValue()));
            if(a <= 0)
            {
                a = 0;
            }
            else
            {
                THROTTLESPINNER.setValue(--a);
            }
            FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
            myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(YAWSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
            myWritter.close();
            process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
            //should integrate when send button is triggered
            AutomaticControlLog.setText("Sending to client:\n\n" +
                    "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                    "Roll value: " + ROLLSPINNER.getValue() + "\n" +
                    "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                    "Throttle value" + THROTTLESPINNER.getValue() + "\n");
        }catch(Exception except) {
            System.out.println("Exception Raised" + except.toString());
        }
```

```
}
else if (keyCode == KeyEvent.VK_LEFT) {
   try{
      int a = Integer.valueOf(String.valueOf(ROLLSPINNER.getValue()));
      if(a <= 0)
      {
         a = 0;
      }
      else
      {
         ROLLSPINNER.setValue(--a);
      }
      FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
      myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
      myWritter.write("|");
      myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
      myWritter.write("|");
      myWritter.write(String.valueOf(YAWSPINNER.getValue()));
      myWritter.write("|");
      myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
      myWritter.close();
      process = new ProcessBuilder("\\src\\com\\codebind\\serverSender.bat").start();
      AutomaticControlLog.setText("Sending to client:\n\n" +
            "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
            "Roll value: " + ROLLSPINNER.getValue() + "\n" +
            "Yaw value: " + YAWSPINNER.getValue() + "\n" +
            "Throttle value" + THROTTLESPINNER.getValue() + "\n");
   }catch(Exception except) {
      System.out.println("Exception Raised" + except.toString());
   }

else if (keyCode == KeyEvent.VK_RIGHT) {
   try{
      int a = Integer.valueOf(String.valueOf(ROLLSPINNER.getValue()));
      if(a >= 100)
      {
```

```
            a = 100;
        }
        else
        {
            ROLLSPINNER.setValue(++a);
        }
        FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
        myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(YAWSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
        myWritter.close();
        process = new ProcessBuilder("\\src\\com\\codebind\\serverSender.bat").start();
        AutomaticControlLog.setText("Sending to client:\n\n" +
                "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                "Roll value: " + ROLLSPINNER.getValue() + "\n" +
                "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                "Throttle value" + THROTTLESPINNER.getValue() + "\n");
    }catch(Exception except) {
        System.out.println("Exception Raised" + except.toString());
    }
}
else if (keyCode == KeyEvent.VK_W) {
    try{
        int a = Integer.valueOf(String.valueOf(PITCHSPINNER.getValue()));
        if(a >= 100)
        {
            a = 100;
        }
        else
        {
            PITCHSPINNER.setValue(++a);
```

```
        }
        FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
        myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(YAWSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
        myWritter.close();
        process = new ProcessBuilder("\\src\\com\\codebind\\serverSender.bat").start();
        AutomaticControlLog.setText("Sending to client:\n\n" +
                "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                "Roll value: " + ROLLSPINNER.getValue() + "\n" +
                "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                "Throttle value" + THROTTLESPINNER.getValue() + "\n");
    }catch(Exception except) {
        System.out.println("Exception Raised" + except.toString());
    }
}
else if (keyCode == KeyEvent.VK_S) {
    try{
        int a = Integer.valueOf(String.valueOf(PITCHSPINNER.getValue()));
        if(a <= 0)
        {
            a = 0;
        }
        else
        {
            PITCHSPINNER.setValue(--a);
        }

        FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
        myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
        myWritter.write("|");
```

```
            myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(YAWSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
            myWritter.close();
            process = new ProcessBuilder("src\\com\\codebind\\serverSender.bat").start();
            AutomaticControlLog.setText("Sending to client:\n\n" +
                    "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                    "Roll value: " + ROLLSPINNER.getValue() + "\n" +
                    "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                    "Throttle value" + THROTTLESPINNER.getValue() + "\n");
        }catch(Exception except) {
            System.out.println("Exception Raised" + except.toString());
        }
    }
    else if (keyCode == KeyEvent.VK_A) {
        try{
            int a = Integer.valueOf(String.valueOf(YAWSPINNER.getValue()));
            if(a <= 0)
            {
               a = 0;
            }
            else
            {
               YAWSPINNER.setValue(--a);
            }
            FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
            myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(YAWSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
```

```
        myWritter.close();
        process = new ProcessBuilder("\\src\\com\\codebind\\serverSender.bat").start();
        AutomaticControlLog.setText("Sending to client:\n\n" +
                "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                "Roll value: " + ROLLSPINNER.getValue() + "\n" +
                "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                "Throttle value" + THROTTLESPINNER.getValue() + "\n");
    }catch(Exception except) {
        System.out.println("Exception Raised" + except.toString());
    }
}
else if (keyCode == KeyEvent.VK_D) {
    try{
        int a = Integer.valueOf(String.valueOf(YAWSPINNER.getValue()));
        if(a >= 100)
        {
            a = 100;
        }
        else
        {
            YAWSPINNER.setValue(++a);
        }
        FileWriter myWritter = new FileWriter("\\src\\com\\codebind\\log.txt");
        myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(YAWSPINNER.getValue()));
        myWritter.write("|");
        myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
        myWritter.close();
        process = new ProcessBuilder("\\src\\com\\codebind\\serverSender.bat").start();
        AutomaticControlLog.setText("Sending to client:\n\n" +
                "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                "Roll value: " + ROLLSPINNER.getValue() + "\n" +
```

```
                "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                "Throttle value" + THROTTLESPINNER.getValue() + "\n");
        }catch(Exception except) {
            System.out.println("Exception Raised" + except.toString());
        }

    }

    if(connectionRequired == 1)
    {
        System.out.println(PITCHSPINNER.getValue());
        System.out.println(ROLLSPINNER.getValue());
        System.out.println(YAWSPINNER.getValue());
        System.out.println(THROTTLESPINNER.getValue());
        try{
            FileWriter myWritter = new FileWriter("src\\com\\codebind\\log.txt");
            myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(YAWSPINNER.getValue()));
            myWritter.write("|");
            myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
            myWritter.close();
            process = new ProcessBuilder("src\\com\\codebind\\serverSender.bat").start();
            quadcopterLogTextArea.setText("Sending to client:\n\n" +
                "Pitch value: " + PITCHSPINNER.getValue() + "\n" +
                "Roll value: " + ROLLSPINNER.getValue() + "\n" +
                "Yaw value: " + YAWSPINNER.getValue() + "\n" +
                "Throttle value" + THROTTLESPINNER.getValue() + "\n");
        }catch(Exception except) {
            System.out.println("Exception Raised" + except.toString());
        }
    }}
});
```

Acest algoritm va incrementa şi decrementa valorile mișcărilor pe care drona trebuie să le facă la apăsarea butoanelor setate. După ce incrementează şi decrementează aceste valori (ca la o radio telecomandă), apelează aceeași logică, scrie aceste valori in fișierul text, fișier ce este interfața între codul Java şi script, după care apelează scriptul de Python cu ajutorul scriptului batch.

4.3 Dezvoltarea algoritmilor de interfațare dintre baza de date și microcontroler utilizând un SBC.

Microcontrolerul dronei nu are posibilitatea să acceseze baza de date fără a avea o interfață care să-i ofere acest lucru. Deci am ales să adaug la bordul dronei un computer SBC. Raspberry Pi 4 este un SBC destul de puternic, un computer ce poate realiza interfațarea microcontrolerului cu baza de date. Cu ajutorul manipulării pinilor GPIO a SBC-ului Raspberry Pi 4, putem transforma aceste date in impulsuri de durată sau lungime variabilă.

Pe Raspberry Pi am implementat următorul algoritm

```
from firebase import firebase
from datetime import datetime,timezone
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
GPIO.setup(12, GPIO.OUT)
GPIO.setup(13, GPIO.OUT)
GPIO.setup(18, GPIO.OUT)
GPIO.setup(19, GPIO.OUT)
pYaw = GPIO.PWM(12, 50)
pRoll = GPIO.PWM(13, 50)
pPitch = GPIO.PWM(18, 50)
pThrottle = GPIO.PWM(19, 50)
pYaw.start(5)
pRoll.start(5)
pPitch.start(5)
pThrottle.start(5)
firebase = firebase.FirebaseApplication("https://database-licenta-default-rtdb.europe-
west1.firebasedatabase.app/", None)
num = 0
```

```
read = firebase.get('/fir-database-3fb8a-default-rtdb', 'Customer')
while(True):
    read = firebase.get('/fir-database-3fb8a-default-rtdb', 'Customer')
    print(read)
    num = [0, 0, 0, 0]
    j = 0
    usableData = 0
    if read is not None:
        for i in str(read):
            if i == ':':
                usableData = 1
            if i == ',':
                usableData = 0
            if i == '|' and usableData == 1:
                j = j + 1
            elif i.isdigit() and usableData == 1:
                try:
                    num[j] = num[j] * 10
                    num[j] = num[j] + int(i)
                except IndexError:   pass
            else:
                continue
    print("yaw:" ,  num[0])
    print("roll:" ,  num[1])
    print("pitch:" , num[2])
    print("throttle:", int(num[3]/10))
    pYaw.ChangeDutyCycle((5*num[0]+500)/100)
    pRoll.ChangeDutyCycle((5*num[1]+500)/100)
    pPitch.ChangeDutyCycle((5*num[2]+500)/100)
    pThrottle.ChangeDutyCycle((5*num[3]+500)/100)
```

Acest script va activa pinii de PWM a plăcii Raspberry Pi. În a doua etapă se va conecta la Firebase şi va citi datele în format text. Extrage datele redundante (numerele de care avem nevoie) şi aplicăm un filtru. Acest filtru va transforma datele mele în impulsuri care au o durată cuprinsă între 1000uS şi 2000uS, impulsuri ce vor fi receptate ca întreruperi de către microcontroler.

4.4 Algoritmul de zbor a dronei

După ce microcontrolerul primeşte comenzile de la Raspberry Pi sub forma de impulsuri, acesta le va trata ca pe întreruperi şi cu ajutorul timerelor el va calcula durata acestora. Ca procesul să fie unul cât mai rapid, trebuie sa apelăm la un serviciu de întreruperi.

```
ISR(PCINT0_vect) {
        timp_prezent = micros();

        // Channel 1 -------------------------------------------------
        if (PINB & B00000001) {
            if (stare_anterioara[CANAL1] == LOW) {
                stare_anterioara[CANAL1] = HIGH;
                timer[CANAL1] = timp_prezent;
            }
        } else if (stare_anterioara[CANAL1] == HIGH) {
            stare_anterioara[CANAL1] = LOW;
            lungime_impuls[CANAL1] = timp_prezent - timer[CANAL1];
        }

        // Channel 2 -------------------------------------------------
        if (PINB & B00000010) {
            if (stare_anterioara[CANAL2] == LOW) {
                stare_anterioara[CANAL2] = HIGH;
                timer[CANAL2] = timp_prezent;
            }
        } else if (stare_anterioara[CANAL2] == HIGH) {
            stare_anterioara[CANAL2] = LOW;
            lungime_impuls[CANAL2] = timp_prezent - timer[CANAL2];
        }

        // Channel 3 -------------------------------------------------
        if (PINB & B00000100) {
            if (stare_anterioara[CANAL3] == LOW) {
                stare_anterioara[CANAL3] = HIGH;
                timer[CANAL3] = timp_prezent;
            }
        } else if (stare_anterioara[CANAL3] == HIGH) {
            stare_anterioara[CANAL3] = LOW;
            lungime_impuls[CANAL3] = timp_prezent - timer[CANAL3];
        }

        // Channel 4 -------------------------------------------------
        if (PINB & B00001000) {
            if (stare_anterioara[CANAL4] == LOW) {
                stare_anterioara[CANAL4] = HIGH;
                timer[CANAL4] = timp_prezent;
            }
        } else if (stare_anterioara[CANAL4] == HIGH) {
            stare_anterioara[CANAL4] = LOW;
            lungime_impuls[CANAL4] = timp_prezent - timer[CANAL4];
        }
}
```

Figura 25, Algoritm de detecţie şi calcul al întreruperilor.

Pentru fiecare pin în parte, dacă a fost detectă o tranziţie de la LOW la HIGH atunci se reţine când a fost făcută această tranziţie. După ce revine în LOW din starea HIGH, calculează cât timp a trecut. Foloseşte acest timp pentru a calcula referinţa sistemului. Algoritmul de reglare folosit este un algoritm proporţional-integral-derivativ, iar factorii de amplificare pentru fiecare componenta (proporţională, integrală sau derivativă) se pot alege in mod experimental după timpul de răspuns şi calitatea reglării răspunsului la semnale de comandă şi semnale perturbatoare.

După setup-ul codului urmează partea de execuţie ciclică. Aici începe algoritmul de reglare propriu-zis a dronei.

```
void loop() {
    readSensor();
    calculateAngles();
    calculateSetPoints();
    calculateErrors();
    pidController();
    applyMotorSpeed();
}
```

Figura 26, Execuţia ciclică al algoritmului de zbor.

Prima dată se colectează datele de la senzori şi accelerometru. După care calculăm unghiurile folosind datele furnizate de la accelerometru. Se calculează punctele de referinţă folosind valorile de timp ale impulsurilor furnizate de la Raspberry Pi. Calculam erorile comparând mișcările unghiulare cu punctele de referință. Aplicăm reglarea PID utilizând referinţele şi erorile. Iar în final aplicăm ficărui ESC in parte semnale PWM pentru a putea controla viteza fiecărui rotor.

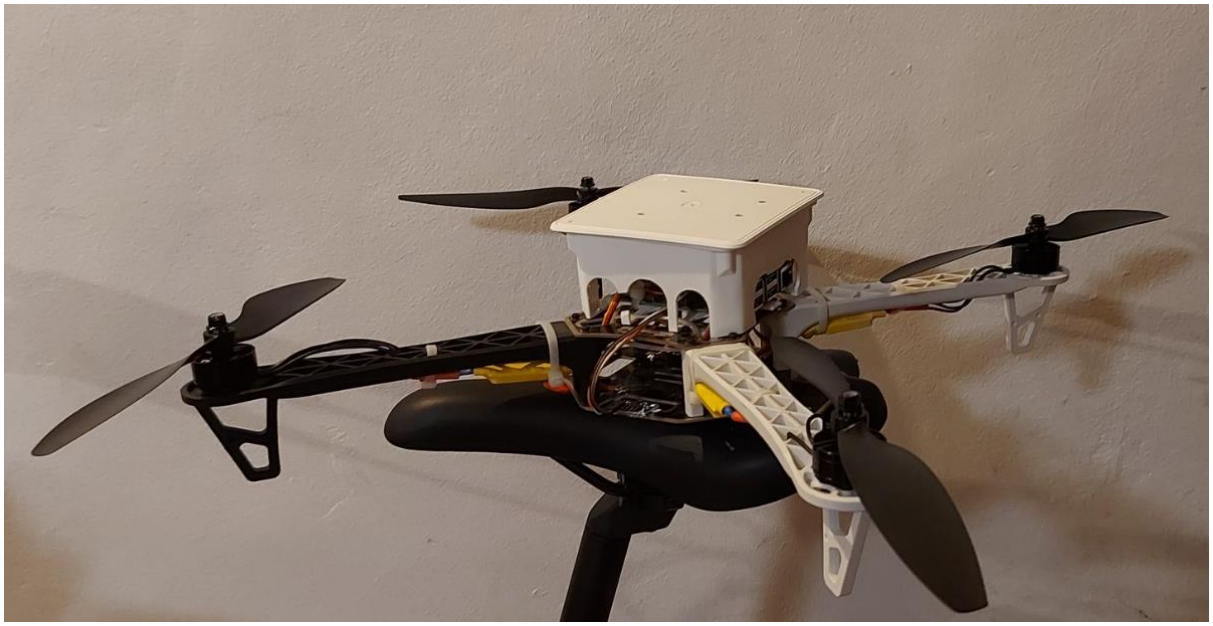


Figura 27, Drona realizată.

5 CONCLUZII

În încheiere, doresc a spune ca o asemenea abordare a sistemului de zbor a dronei, în combinație cu senzori ce pot fi integrați modular într-un astfel de sistem, pot duce la salvarea a multor vieți, operatorii pot prelua măsurători din medii sigure, nepunând viața și integritatea lor în pericol. Un asemenea procedeu elimină mai ales componenta de stres, măsurătorile și operațiunile executate având un randament și o acuitate mult mai bună.

Desigur, pot spune ca scopul lucrării a fost atins, dar în viitor implementarea unui sistem comandat de către un program automat cu inteligenta artificială poate înlătura toată componenta umană, evitând astfel riscul de erori ce pot apărea din cauza acestei componente. Acest concept este unul din ideile ce doresc să le implementez în viitor, completările de vigoare și reînnoirea sistemului vor fi o responsabilitate ce mi-o v-oi asuma cu mare drag.

6 BIBLIOGRAFIE

[1] - Despre limbajul C

https://www.emertxe.com/c-programming/why-is-c-the-most-preferred-language-for-embedded-systems/

[2] - Datasheet Atmega328P

https://pdf1.alldatasheet.com/datasheet-pdf/view/313218/ATMEL/ATmega328P.html

[3]- Despre Tattu 2305 2405KV

https://www.genstattu.com/ta-motor-2305-2450kv.html

[4] - MPU6050 Datasheet

https://www.arrow.com/en/products/mpu-6050/invensense?region=nac

[5] – Despre IntelliJ

https://www.tutorialspoint.com/intellij_idea/index.htm

[6] - Despre Python

https://www.python.org/doc/essays/blurb/

[7] – MQ2 Datasheet

https://www.pololu.com/file/0J309/MQ2.pdf

[8] – Despre interfaţarea MPU6050 cu Arduino

https://lastminuteengineers.com/mpu6050-accel-gyro-arduino-tutorial/

[9] – Despre circuitul de detecţie a radiaţiilor

https://community.element14.com/challenges-projects/element14-presents/b/blog/posts/a-homebrew-geiger-counter-circuit

[10] – Despre RaspberryPi

https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf

[11] – Despre quadcoptere

https://docplayer.net/21716235-Quadcopters-presented-by-andrew-depriest.html

[12] – Despre Firebase

https://ik4.es/ro/firebase-que-es-y-como-funciona/

[13] – Modelling and control of quadcopter (Teppo Luukkonen)

https://sal.aalto.fi/publications/pdf-files/eluu11_public.pdf

A CODUL SURSĂ

Codul sursă a aplicației Java

```
package com.codebind;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.FileWriter;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
import static com.sun.java.accessibility.util.AWTEventMonitor.addKeyListener;
public class App {
private JTabbedPane tabbedPane1;
private JPanel panel1;
private JPanel TAB_CONNECTION;
private JPanel TAB_CONTROL;
private JCheckBox connectionStatusCheckBox;
private JButton connectButton;
private JPasswordField passwordField1;
private JTextField textField1;
private JTextArea logTextTextArea;
private JLabel LABEL1;
private JLabel LABEL_PASSWORD;
private JLabel LABEL_USERNAME;
private JTabbedPane Control_Tab;
private JRadioButton contollerEnableRadioButton;
private JRadioButton standaloneConfigurationRadioButton;
private JLabel LABEL_SETTINGS;
private JPanel ControlEnv;
private JPanel Control_Settings;
private JLabel connectionStatusLabel;
private JSpinner PITCHSPINNER;
private JSpinner THROTTLESPINNER;
private JSpinner YAWSPINNER;
private JSpinner ROLLSPINNER;
```

```
private JButton sendButton;
private JTextArea quadcopterLogTextArea;
private JLabel PITCH;
private JLabel ROLL;
private JLabel YAW;
private JButton panicButton;
private JCheckBox continousSenderCheckBox;
private JTextArea AutomaticControlLog;
private int connectionRequired = 0;
private int YawContainer = 0;
private int PitchContainer = 0;
private int RollContainer = 0;
Process process;
public App() {
connectionStatusCheckBox.setEnabled(false);
standaloneConfigurationRadioButton.setSelected(true);
Timer timer = new Timer(200,new ActionListener(){
public void actionPerformed(ActionEvent e)
{

if(standaloneConfigurationRadioButton.isSelected())
{
if(connectionRequired == 1 && continousSenderCheckBox.isSelected())
{
quadcopterLogTextArea.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue()  +"\n"+
"Roll value: "  + ROLLSPINNER.getValue()   +"\n"+
"Yaw value: "   + YAWSPINNER.getValue()    +"\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n" +
"\n\n\n"+
"Gas level:" + "\n"+
"Radiation Meter: " + "\n");
try{
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
```

```
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
quadcopterLogTextArea.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue()  +"\n"+
"Roll value: "  + ROLLSPINNER.getValue()   +"\n"+
"Yaw value: "   + YAWSPINNER.getValue()    +"\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
}
//END
}
});
timer.start();
connectButton.addActionListener(new ActionListener() {
@Override
public void actionPerformed(ActionEvent e) {
String username = new String(textField1.getText());
String password = new String(passwordField1.getPassword());
if(username.equals("user"))
{
logTextTextArea.setText("got here");
if(password.equals("drone"))
{
connectionRequired = 1;
connectionStatusCheckBox.setEnabled(true);
```

```
connectionStatusLabel.setForeground(Color.GREEN);
if(!connectionStatusCheckBox.isSelected())
{
connectionStatusCheckBox.doClick();
}
else
{
//pass
}
connectionStatusCheckBox.setEnabled(false);
logTextTextArea.setText("Username and Password are correct!\n");
}
else
{
logTextTextArea.setText("Username or Password are incorrect!\n");
connectionRequired = 0;
if(connectionStatusCheckBox.isSelected())
{
connectionStatusCheckBox.setEnabled(true);
connectionStatusCheckBox.doClick();
connectionStatusCheckBox.setEnabled(false);
}
connectionStatusLabel.setForeground(Color.BLACK);
}
}
else
{
logTextTextArea.setText("Username or Password are incorrect!\n");
connectionRequired = 0;
if(connectionStatusCheckBox.isSelected())
{
connectionStatusCheckBox.setEnabled(true);
connectionStatusCheckBox.doClick();
connectionStatusCheckBox.setEnabled(false);
}
```

```
connectionStatusLabel.setForeground(Color.BLACK);

}
}
});
sendButton.addActionListener(new ActionListener() {
@Override
public void actionPerformed(ActionEvent e) {
if(connectionRequired == 1)
{
System.out.println(PITCHSPINNER.getValue());
System.out.println(ROLLSPINNER.getValue());
System.out.println(YAWSPINNER.getValue());
System.out.println(THROTTLESPINNER.getValue());

try{

FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");

myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));

myWritter.close();

process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
//should integrate when send button is triggered
quadcopterLogTextArea.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
```

```
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");


}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
}
});
contollerEnableRadioButton.addKeyListener(new KeyAdapter() {
@Override public void keyPressed(KeyEvent e) {
int keyCode = e.getKeyCode();
if (keyCode == KeyEvent.VK_UP && connectionRequired == 1) {
System.out.println("Up Arrrow-Key is pressed!");
try{
int a = Integer.valueOf(String.valueOf(THROTTLESPINNER.getValue()));
if(a >= 100 )
{
a = 100;
}
else
{
THROTTLESPINNER.setValue(++a);
}
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
```

```
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
else if (keyCode == KeyEvent.VK_DOWN) {
System.out.println("Down Arrrow-Key is pressed!");
try{
int a = Integer.valueOf(String.valueOf(THROTTLESPINNER.getValue()));
if(a <= 0)
{
a = 0;
}
else
{
THROTTLESPINNER.setValue(--a);
}
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
```

```
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
else if (keyCode == KeyEvent.VK_LEFT) {
try{
int a = Integer.valueOf(String.valueOf(ROLLSPINNER.getValue()));
if(a <= 0)
{
a = 0;
}
else
{
ROLLSPINNER.setValue(--a);
}
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
```

```
System.out.println("Exception Raised" + except.toString());
}
}
else if (keyCode == KeyEvent.VK_RIGHT) {
try{
int a = Integer.valueOf(String.valueOf(ROLLSPINNER.getValue()));
if(a >= 100)
{
a = 100;
}
else
{
ROLLSPINNER.setValue(++a);
}
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
else if (keyCode == KeyEvent.VK_W) {
```

```
try{
int a = Integer.valueOf(String.valueOf(PITCHSPINNER.getValue()));
if(a >= 100)
{
a = 100;
}
else
{
PITCHSPINNER.setValue(++a);
}
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
//should integrate when send button is triggered
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
else if (keyCode == KeyEvent.VK_S) {
try{
int a = Integer.valueOf(String.valueOf(PITCHSPINNER.getValue()));
if(a <= 0)
```

```
{
a = 0;
}
else
{
PITCHSPINNER.setValue(--a);
}
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
else if (keyCode == KeyEvent.VK_A) {
try{
int a = Integer.valueOf(String.valueOf(YAWSPINNER.getValue()));
if(a <= 0)
{
a = 0;
}
else
```

```
{
YAWSPINNER.setValue(--a);
}

FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
else if (keyCode == KeyEvent.VK_D) {
try{
int a = Integer.valueOf(String.valueOf(YAWSPINNER.getValue()));
if(a >= 100)
{
a = 100;
}
else
{
YAWSPINNER.setValue(++a);
}
```

```
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
AutomaticControlLog.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
if(connectionRequired == 1)
{
System.out.println(PITCHSPINNER.getValue());
System.out.println(ROLLSPINNER.getValue());
System.out.println(YAWSPINNER.getValue());
System.out.println(THROTTLESPINNER.getValue());
try{
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
```

```
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
quadcopterLogTextArea.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
}
});
panicButton.addActionListener(new ActionListener() {
@Override
public void actionPerformed(ActionEvent e) {
//put yaw and spin to 0
PITCHSPINNER.setValue(0);
YAWSPINNER.setValue(0);
ROLLSPINNER.setValue(0);
if(connectionRequired == 1)
{
System.out.println(PITCHSPINNER.getValue());
System.out.println(ROLLSPINNER.getValue());
System.out.println(YAWSPINNER.getValue());
System.out.println(THROTTLESPINNER.getValue());
quadcopterLogTextArea.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue()  +"\n"+
"Roll value: "  + ROLLSPINNER.getValue()   +"\n"+
"Yaw value: "   + YAWSPINNER.getValue()    +"\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
try{
FileWriter myWritter = new FileWriter("D:\\projects\\AppConection\\src\\com\\codebind\\log.txt");
myWritter.write(String.valueOf(PITCHSPINNER.getValue()));
```

```
myWritter.write("|");
myWritter.write(String.valueOf(ROLLSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(YAWSPINNER.getValue()));
myWritter.write("|");
myWritter.write(String.valueOf(THROTTLESPINNER.getValue()));
myWritter.close();
process = new
ProcessBuilder("D:\\projects\\AppConection\\src\\com\\codebind\\serverSender.bat").start();
quadcopterLogTextArea.setText("Sending to client:\n\n" +
"Pitch value: " + PITCHSPINNER.getValue() + "\n" +
"Roll value: " + ROLLSPINNER.getValue() + "\n" +
"Yaw value: " + YAWSPINNER.getValue() + "\n" +
"Throttle value" + THROTTLESPINNER.getValue() + "\n");
}catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
}
});
try {
process.destroy();
}
catch(Exception except) {
System.out.println("Exception Raised" + except.toString());
}
}
public static void main(String[] args)
{
JFrame frame = new JFrame("Quadcopter Remote Control GUI");
frame.setContentPane(new App().panel1);
frame.setSize(1800,900);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.pack();
frame.setVisible(true);
```

```
}
}
```

Scriptul XML a interfeței grafice

```
<?xml version="1.0" encoding="UTF-8"?>

<form xmlns="http://www.intellij.com/uidesigner/form/" version="1" bind-to-class="com.codebind.App">

  <grid id="27dc6" binding="panel1" default-binding="true" layout-manager="GridLayoutManager" row-count="2" column-count="1" same-size-horizontally="false" same-size-vertically="false" hgap="-1" vgap="-1">

    <margin top="0" left="0" bottom="0" right="0"/>

    <constraints>

      <xy x="20" y="20" width="797" height="553"/>

    </constraints>

    <properties/>

    <border type="none"/>

    <children>

      <tabbedpane id="e0f55" binding="tabbedPane1" default-binding="true">

        <constraints>

          <grid row="0" column="0" row-span="1" col-span="1" vsize-policy="3" hsize-policy="3" anchor="0" fill="3" indent="0" use-parent-layout="false">

            <preferred-size width="200" height="200"/>

          </grid>

        </constraints>
```

<properties/>

<border type="none"/>

<children>

<grid id="57100" binding="TAB_CONNECTION" layout-manager="GridLayoutManager" row-count="6" column-count="5" same-size-horizontally="false" same-size-vertically="false" hgap="-1" vgap="-1">

<margin top="0" left="0" bottom="0" right="0"/>

<constraints>

<tabbedpane title="Connection"/>

</constraints>

<properties/>

<border type="none"/>

<children>

<component id="834c5" class="javax.swing.JLabel" binding="LABEL1">

<constraints>

<grid row="0" column="0" row-span="1" col-span="1" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="Please provide internal username and password to preform the connection to the data base."/>

</properties>

</component>

```
        <hspacer id="cc9e5">
          <constraints>
            <grid row="0" column="1" row-span="1" col-span="1" vsize-policy="1" hsize-policy="6" anchor="0" fill="1" indent="0" use-parent-layout="false"/>
          </constraints>
        </hspacer>
        <component id="758e8" class="javax.swing.JButton" binding="connectButton" default-binding="true">
          <constraints>
            <grid row="5" column="2" row-span="1" col-span="3" vsize-policy="0" hsize-policy="3" anchor="0" fill="1" indent="0" use-parent-layout="false"/>
          </constraints>
          <properties>
            <background color="-6557030"/>
            <text value="Connect"/>
          </properties>
        </component>
        <vspacer id="5054d">
          <constraints>
            <grid row="2" column="2" row-span="3" col-span="3" vsize-policy="6" hsize-policy="1" anchor="0" fill="2" indent="0" use-parent-layout="false"/>
          </constraints>
        </vspacer>
```

<component id="5525d" class="javax.swing.JPasswordField" binding="passwordField1" default-binding="true">

<constraints>

<grid row="5" column="0" row-span="1" col-span="1" vsize-policy="0" hsize-policy="6" anchor="8" fill="1" indent="0" use-parent-layout="false">

<preferred-size width="150" height="-1"/>

</grid>

</constraints>

<properties/>

</component>

<component id="26720" class="javax.swing.JLabel" binding="LABEL_PASSWORD">

<constraints>

<grid row="4" column="0" row-span="1" col-span="1" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="Password"/>

</properties>

</component>

<component id="3032c" class="javax.swing.JLabel" binding="LABEL_USERNAME">

<constraints>

<grid row="2" column="0" row-span="1" col-span="1" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

```
        </constraints>
        <properties>
          <text value="Username"/>
        </properties>
      </component>
      <grid id="b2476" layout-manager="GridLayoutManager" row-count="1" column-count="1" same-size-horizontally="false" same-size-vertically="false" hgap="-1" vgap="-1">
        <margin top="0" left="0" bottom="0" right="0"/>
        <constraints>
          <grid row="1" column="0" row-span="1" col-span="5" vsize-policy="3" hsize-policy="3" anchor="0" fill="3" indent="0" use-parent-layout="false"/>
        </constraints>
        <properties/>
        <border type="none"/>
        <children>
          <component id="2b320" class="javax.swing.JTextArea" binding="logTextTextArea" default-binding="true">
            <constraints>
              <grid row="0" column="0" row-span="1" col-span="1" vsize-policy="6" hsize-policy="6" anchor="0" fill="3" indent="0" use-parent-layout="false">
                <preferred-size width="150" height="50"/>
              </grid>
            </constraints>
```

<properties>

<text value="LogText"/>

</properties>

</component>

</children>

</grid>

<component id="9689e" class="javax.swing.JTextField" binding="textField1" default-binding="true">

<constraints>

<grid row="3" column="0" row-span="1" col-span="1" vsize-policy="0" hsize-policy="6" anchor="8" fill="1" indent="0" use-parent-layout="false">

<preferred-size width="150" height="-1"/>

</grid>

</constraints>

<properties/>

</component>

<component id="76cd7" class="javax.swing.JLabel" binding="connectionStatusLabel">

<constraints>

<grid row="0" column="4" row-span="1" col-span="1" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="Connection Status"/>

```
          </properties>
        </component>
        <component id="e152" class="javax.swing.JCheckBox"
binding="connectionStatusCheckBox" default-binding="true">
          <constraints>
            <grid row="0" column="3" row-span="1" col-span="1" vsize-policy="0" hsize-policy="3"
anchor="8" fill="0" indent="0" use-parent-layout="false"/>
          </constraints>
          <properties>
            <text value=""/>
          </properties>
        </component>
      </children>
    </grid>
    <grid id="ca31f" binding="TAB_CONTROL" layout-manager="GridLayoutManager" row-
count="2" column-count="1" same-size-horizontally="false" same-size-vertically="false" hgap="-1"
vgap="-1">
      <margin top="0" left="0" bottom="0" right="0"/>
      <constraints>
        <tabbedpane title="Control"/>
      </constraints>
      <properties/>
      <border type="none"/>
```

<children>

<grid id="b4bdf" layout-manager="GridLayoutManager" row-count="1" column-count="2" same-size-horizontally="false" same-size-vertically="false" hgap="-1" vgap="-1">

<margin top="0" left="0" bottom="0" right="0"/>

<constraints>

<grid row="1" column="0" row-span="1" col-span="1" vsize-policy="3" hsize-policy="3" anchor="0" fill="3" indent="0" use-parent-layout="false"/>

</constraints>

<properties/>

<border type="none"/>

<children>

<tabbedpane id="90b2c" binding="Control_Tab">

<constraints>

<grid row="0" column="1" row-span="1" col-span="1" vsize-policy="3" hsize-policy="3" anchor="0" fill="3" indent="0" use-parent-layout="false">

<preferred-size width="200" height="200"/>

</grid>

</constraints>

<properties/>

<border type="none"/>

<children>

<grid id="a77ee" binding="ControlEnv" layout-manager="GridLayoutManager" row-count="12" column-count="4" same-size-horizontally="false" same-size-vertically="false" hgap="-1" vgap="-1">

<margin top="0" left="0" bottom="0" right="0"/>

<constraints>

<tabbedpane title="Control Environment"/>

</constraints>

<properties/>

<border type="none"/>

<children>

<component id="a9469" class="javax.swing.JSpinner" binding="PITCHSPINNER">

<constraints>

<grid row="4" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="6" anchor="8" fill="1" indent="0" use-parent-layout="false"/>

</constraints>

<properties/>

</component>

<component id="e5e7" class="javax.swing.JSpinner" binding="THROTTLESPINNER">

<constraints>

<grid row="10" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="6" anchor="8" fill="1" indent="0" use-parent-layout="false"/>

</constraints>

<properties/>

</component>

<component id="38147" class="javax.swing.JSpinner" binding="YAWSPINNER">

<constraints>

<grid row="8" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="6" anchor="8" fill="1" indent="0" use-parent-layout="false"/>

</constraints>

<properties/>

</component>

<component id="fa72e" class="javax.swing.JSpinner" binding="ROLLSPINNER">

<constraints>

<grid row="6" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="6" anchor="8" fill="1" indent="0" use-parent-layout="false"/>

</constraints>

<properties/>

</component>

<component id="25113" class="javax.swing.JTextArea" binding="quadcopterLogTextArea" default-binding="true">

<constraints>

<grid row="0" column="0" row-span="1" col-span="4" vsize-policy="6" hsize-policy="6" anchor="0" fill="3" indent="0" use-parent-layout="false">

<preferred-size width="150" height="50"/>

</grid>

</constraints>

<properties>

<text value="Quadcopter Log"/>

```
      </properties>
    </component>
    <component id="c11ad" class="javax.swing.JLabel" binding="PITCH">
      <constraints>
        <grid row="3" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>
      </constraints>
      <properties>
        <text value="PITCH"/>
      </properties>
    </component>
    <component id="bf450" class="javax.swing.JLabel" binding="ROLL">
      <constraints>
        <grid row="5" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>
      </constraints>
      <properties>
        <text value="ROLL"/>
      </properties>
    </component>
    <component id="9083" class="javax.swing.JLabel" binding="YAW">
      <constraints>
```

<grid row="7" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="YAW"/>

</properties>

</component>

<component id="b5d8e" class="javax.swing.JLabel">

<constraints>

<grid row="9" column="0" row-span="1" col-span="2" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="THROTTLE"/>

</properties>

</component>

<component id="4c01b" class="javax.swing.JButton" binding="sendButton" default-binding="true">

<constraints>

<grid row="11" column="0" row-span="1" col-span="4" vsize-policy="0" hsize-policy="3" anchor="0" fill="1" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="Send"/>

</properties>

</component>

<component id="abea4" class="javax.swing.JButton" binding="panicButton">

<constraints>

<grid row="1" column="1" row-span="1" col-span="2" vsize-policy="0" hsize-policy="3" anchor="0" fill="1" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<background color="-864160"/>

<text value="Panic Button"/>

</properties>

</component>

<vspacer id="c6717">

<constraints>

<grid row="2" column="2" row-span="1" col-span="1" vsize-policy="6" hsize-policy="1" anchor="0" fill="2" indent="0" use-parent-layout="false"/>

</constraints>

</vspacer>

<component id="3e0e3" class="javax.swing.JCheckBox" binding="continousSenderCheckBox" default-binding="true">

<constraints>

<grid row="2" column="1" row-span="1" col-span="1" vsize-policy="0" hsize-policy="3" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="Continous Sender"/>

</properties>

</component>

</children>

</grid>

<grid id="8438c" binding="Control_Settings" layout-manager="GridLayoutManager" row-count="1" column-count="1" same-size-horizontally="false" same-size-vertically="false" hgap="-1" vgap="-1">

<margin top="0" left="0" bottom="0" right="0"/>

<constraints>

<tabbedpane title="Control Settings"/>

</constraints>

<properties/>

<border type="none"/>

<children>

<grid id="6a81c" layout-manager="GridLayoutManager" row-count="5" column-count="2" same-size-horizontally="false" same-size-vertically="false" hgap="-1" vgap="-1">

<margin top="0" left="0" bottom="0" right="0"/>

<constraints>

<grid row="0" column="0" row-span="1" col-span="1" vsize-policy="3" hsize-policy="3" anchor="0" fill="3" indent="0" use-parent-layout="false"/>

</constraints>

<properties/>

<border type="none"/>

<children>

<component id="eae35" class="javax.swing.JLabel" binding="LABEL_SETTINGS">

<constraints>

<grid row="1" column="0" row-span="1" col-span="2" vsize-policy="0" hsize-policy="0" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="Settings: Only one condition must be checked"/>

</properties>

</component>

<component id="c00c4" class="javax.swing.JRadioButton" binding="contollerEnableRadioButton" default-binding="true">

<constraints>

<grid row="2" column="0" row-span="1" col-span="2" vsize-policy="0" hsize-policy="3" anchor="8" fill="0" indent="0" use-parent-layout="false"/>

</constraints>

<properties>

<text value="Contoller Enable"/>

```
        </properties>
      </component>
      <component id="c83d8" class="javax.swing.JRadioButton"
binding="standaloneConfigurationRadioButton" default-binding="true">
        <constraints>
          <grid row="3" column="0" row-span="1" col-span="2" vsize-policy="0" hsize-
policy="3" anchor="8" fill="0" indent="0" use-parent-layout="false"/>
        </constraints>
        <properties>
          <text value="Standalone Configuration"/>
        </properties>
      </component>
      <hspacer id="55e58">
        <constraints>
          <grid row="0" column="0" row-span="1" col-span="2" vsize-policy="1" hsize-
policy="6" anchor="0" fill="1" indent="0" use-parent-layout="false"/>
        </constraints>
      </hspacer>
      <component id="3c6f7" class="javax.swing.JTextArea"
binding="AutomaticControlLog">
        <constraints>
          <grid row="4" column="0" row-span="1" col-span="2" vsize-policy="6" hsize-
policy="6" anchor="0" fill="3" indent="0" use-parent-layout="false">
            <preferred-size width="150" height="50"/>
```

```
                    </grid>
                   </constraints>
                   <properties>
                    <text value=""/>
                   </properties>
                  </component>
                 </children>
                </grid>
               </children>
              </grid>
             </children>
            </tabbedpane>
            <vspacer id="f90d1">
             <constraints>
              <grid row="0" column="0" row-span="1" col-span="1" vsize-policy="6" hsize-policy="1" anchor="0" fill="2" indent="0" use-parent-layout="false"/>
             </constraints>
            </vspacer>
           </children>
          </grid>
          <hspacer id="f2c33">
           <constraints>
```

```
                <grid row="0" column="0" row-span="1" col-span="1" vsize-policy="1" hsize-policy="6"
anchor="0" fill="1" indent="0" use-parent-layout="false"/>

              </constraints>

            </hspacer>

          </children>

        </grid>

      </children>

    </tabbedpane>

    <hspacer id="785d6">

      <constraints>

        <grid row="1" column="0" row-span="1" col-span="1" vsize-policy="1" hsize-policy="6"
anchor="0" fill="1" indent="0" use-parent-layout="false"/>

      </constraints>

    </hspacer>

  </children>

 </grid>

</form>
```

Codul Python prentru transmisie a datelor

```
from firebase import firebase
import time
f = open("D:\projects\AppConection\src\com\codebind\log.txt","r")
data_raw = f.read()
data_input = {"data" : data_raw}
print(f.read())
```

```
a = time.time_ns()
print(a)
firebase = firebase.FirebaseApplication("https://database-licenta-default-rtdb.europe-
west1.firebasedatabase.app/", None)
delete = firebase.delete('/fir-database-3fb8a-default-rtdb', 'Customer')
data = data_input
post = firebase.post('/fir-database-3fb8a-default-rtdb/Customer', data)
```

Script batch pentru apelul codului Python

```
@echo off

"C:\Users\User\AppData\Local\Programs\Python\Python310\python.exe"
"D:\projects\AppConection\src\com\codebind\sender.py"

Pause
```

Script Python control citire date din firebase si trimitere PWM către microcontroler

```
from firebase import firebase
from datetime import datetime,timezone
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
GPIO.setup(12, GPIO.OUT)
GPIO.setup(13, GPIO.OUT)
GPIO.setup(18, GPIO.OUT)
GPIO.setup(19, GPIO.OUT)
pYaw = GPIO.PWM(12, 50)
pRoll = GPIO.PWM(13, 50)
pPitch = GPIO.PWM(18, 50)
pThrottle = GPIO.PWM(19, 50)
```

```
pYaw.start(5)
pRoll.start(5)
pPitch.start(5)
pThrottle.start(5)
firebase = firebase.FirebaseApplication("https://database-licenta-default-rtdb.europe-
west1.firebasedatabase.app/", None)
num = 0
read = firebase.get('/fir-database-3fb8a-default-rtdb', 'Customer')
while(True):
    read = firebase.get('/fir-database-3fb8a-default-rtdb', 'Customer')
    print(read)
    num = [0, 0, 0, 0]
    j = 0
    usableData = 0
    if read is not None:
        for i in str(read):
            if i == ':':
                usableData = 1
            if i == ',':
                usableData = 0
            if i == '|' and usableData == 1:
                j = j + 1
            elif i.isdigit() and usableData == 1:
                try:
                    num[j] = num[j] * 10
                    num[j] = num[j] + int(i)
                except IndexError:
                    pass
            else:
                continue
    print("yaw:" ,  num[0])
    print("roll:" ,  num[1])
    print("pitch:" , num[2])
    print("throttle:", int(num[3]/10))
    pYaw.ChangeDutyCycle((5*num[0]+500)/100)
```

```
    pRoll.ChangeDutyCycle((5*num[1]+500)/100)
    pPitch.ChangeDutyCycle((5*num[2]+500)/100)
    pThrottle.ChangeDutyCycle((5*num[3]+500)/100)
    print('\n')
    print('-------------')
    print('\n')
```

Codul C de control a dronei

```
#include <Wire.h>

// ------------------- Definitiile constantelor -----------------

#define STD_PRESENT (unsigned) 1

#define STD_ABSENT  (unsigned) 0

#define CANAL1 0

#define CANAL2 1

#define CANAL3 2

#define CANAL4 3

#define YAW      0

#define PITCH    1

#define ROLL     2

#define THROTTLE 3

#define X          0     // axa X

#define Y          1     // axa Y
```

```
#define Z          2     // axa Z

#define MPU_ADDRESS 0x68  // adresa I2C a mpu6050

#define FREQ        250   // frecvența de ceas

#define SSF_GYRO    65.5  // Factor scalar de sensitivitate (datasheet)

#define OPRIT  0

#define PORNIRE 1

#define RULARE  2

// ---------------- Variabilele receptorului ---------------------------------------

// starea fiecarui canal in timp n-1 (HIGH or LOW)

volatile byte stare_anterioara[4];

// durata impulsului receptionat ca întrerupere pe fiecare canal (neapărat între 1000µs & 2000µs)

volatile unsigned int lungime_impuls[4] = {1500, 1500, 1000, 1500};

// folosit in calcul durata fiecare canal

volatile unsigned long timp_prezent;

volatile unsigned long timer[4]; // Timer of each channel

// configurare control fiecare canal (yaw, pitch, roll, throttle)

int mode_mapping[4];

// ---------------------- Variabile MPU  -------------------------------------

// Valori neprelucrate de la gyro (in °/sec) in ordinea asta: X, Y, Z

int gyro_raw[3] = {0,0,0};

//Media suprareglajelor pe fiecare axa : X, Y, Z

long gyro_offset[3] = {0, 0, 0};
```

```
// unghiuri calculate: X, Y, Z

float gyro_angle[3]  = {0,0,0};

// Valori neprelucrate de la accelerometru (in m/sec²) : X, Y, Z

int acc_raw[3] = {0 ,0 ,0};

// accelerometru calculate in ordine: X, Y, Z

float acc_angle[3] = {0,0,0};

// vector total 3D in m/s²

long acc_total_vector;

// Viteza unghiulara pe: Yaw, Pitch, Roll

float miscari_unghiulare[3] = {0, 0, 0};

/**

* Masuratori reale pe 3 axe calculate din giroscop si accelerometru in aceasta ordine: Yaw, Pitch, Roll

* - Aripa stângă în sus implică o rostogolire pozitivă

* - Nasul în sus implică o înclinare pozitivă

* - Nasul drept implică o rotire pozitivă

*/

float masuratori[3] = {0, 0, 0};

// MPU's temperatura

int temperatura;

boolean initializare;

// ---------------------- Variables for servo signal generation ----------

unsigned int  perioada; // Sampling perioada
```

```
unsigned long loop_timer;

unsigned long acum, diferenta;


unsigned long lungime_impuls_esc1 = 1000,

lungime_impuls_esc2 = 1000,

lungime_impuls_esc3 = 1000,

lungime_impuls_esc4 = 1000;



#define GEYGER_SENSOR_PRESENT   STD_ABSENT

#define GAS_SENSOR_PRESENT          STD_PRESENT


// ------------- Global variables used for PID controller --------------------

float pid_set_points[3] = {0, 0, 0}; // Yaw, Pitch, Roll

// Errors

float errors[3];                    // Erori măsurate (comparativ cu instrucțiunile): [Yaw, Pitch, Roll]

float delta_err[3]      = {0, 0, 0}; // Delta eroare pt   : Yaw, Pitch, Roll

float error_sum[3]      = {0, 0, 0}; // Error sums (pt componenta integrala) : [Yaw, Pitch, Roll]

float previous_error[3] = {0, 0, 0}; // Ultimele erori (Pentru derivativa) : [Yaw, Pitch, Roll]

// PID coefficients

float Kp[3] = {4.0, 1.3, 1.3};    // P coefficient

float Ki[3] = {0.02, 0.04, 0.04}; // I coefficient
```

```
float Kd[3] = {0, 18, 18};        // D coefficient

// ------------------------------------------------------------------------

/**

* Status of the quadcopter:

*   - 0 : OPRIT

*   - 1 : PORNIRE

*   - 2 : RULARE

*

* @var int

*/

int status = OPRIT;

// ------------------------------------------------------------------------

int battery_voltage;

// ------------------------------------------------------------------------

/**

* Setup configuration

*/

void setup() {

#if(GEYGER_SENSOR_PRESENT)

// Start timer geyger

noInterrupts();           // disable all interrupts

pinMode(interruptPinTemp, INPUT_PULLUP);
```

```
pinMode(interruptPinSlanost, INPUT_PULLUP);

TCCR1A = 0;

TCCR1B = 0;

timer1_counter = 31250; //34286;   // preload timer 65536-16MHz/256/2Hz

TCNT1 = timer1_counter;   // preload timer

TCCR1B |= (1 << CS12);    // 256 prescaler

TIMSK1 |= (1 << TOIE1);   // enable timer overflow interrupt

interrupts();             // enable all interrupts

#endif

//endif GEYGER_SENSOR_PRESENT

Wire.begin();

TWBR = 12;

pinMode(13, OUTPUT);

digitalWrite(13, HIGH);

Serial.begin(9600);

DDRD |= B11110000;

setupMpu6050Registers();

calibrateMpu6050();

configureChannelMapping();

PCICR  |= (1 << PCIE0);  // Set PCIE0 to enable PCMSK0 scan

PCMSK0 |= (1 << PCINT0); // Set PCINT0 (digital input 8) to trigger an interrupt on state change

PCMSK0 |= (1 << PCINT1); // Set PCINT1 (digital input 9) to trigger an interrupt on state change
```

```
PCMSK0 |= (1 << PCINT2); // Set PCINT2 (digital input 10)to trigger an interrupt on state change

PCMSK0 |= (1 << PCINT3); // Set PCINT3 (digital input 11)to trigger an interrupt on state change

perioada = (1000000/FREQ) ; // Sampling perioada in µs

loop_timer = micros();

delay(5000);

digitalWrite(13, LOW);

}

void loop() {

readSensor();

calculateAngles();

calculateSetPoints();

calculateErrors();

pidController();

applyMotorSpeed();

}

void applyMotorSpeed() {

while ((acum = micros()) - loop_timer < perioada);

loop_timer = acum;

PORTD |= B11110000;

delay(0.1);

while (PORTD >= 16) {

acum        = micros();
```

```
diferenta = acum - loop_timer;
if (diferenta >= lungime_impuls_esc1) PORTD &= B11101111; // Set pin #4 LOW
if (diferenta >= lungime_impuls_esc2) PORTD &= B11011111; // Set pin #5 LOW
if (diferenta >= lungime_impuls_esc3) PORTD &= B10111111; // Set pin #6 LOW
if (diferenta >= lungime_impuls_esc4) PORTD &= B01111111; // Set pin #7 LOW
}
}
void readSensor() {
Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
Wire.write(0x3B);                    // Trimitere request de pornire
Wire.endTransmission();              // Stop transmisiei
Wire.requestFrom(MPU_ADDRESS,14);    // Citeste 14 bytes de la MPU
while(Wire.available() < 14);
acc_raw[X]  = Wire.read() << 8 | Wire.read();
acc_raw[Y]  = Wire.read() << 8 | Wire.read();
acc_raw[Z]  = Wire.read() << 8 | Wire.read();
temperatura = Wire.read() << 8 | Wire.read();
gyro_raw[X] = Wire.read() << 8 | Wire.read();
gyro_raw[Y] = Wire.read() << 8 | Wire.read();
gyro_raw[Z] = Wire.read() << 8 | Wire.read();
}
void calculateAngles() {
```

```
calculateGyroAngles();

calculateAccelerometerAngles();

if (initializare) {

gyro_angle[X] = gyro_angle[X] * 0.9996 + acc_angle[X] * 0.0004;

gyro_angle[Y] = gyro_angle[Y] * 0.9996 + acc_angle[Y] * 0.0004;

} else {

resetGyroAngles();

initializare = true;

}

masuratori[ROLL]  = masuratori[ROLL]  * 0.9 + gyro_angle[X] * 0.1;

masuratori[PITCH] = masuratori[PITCH] * 0.9 + gyro_angle[Y] * 0.1;

masuratori[YAW]   = -gyro_raw[Z] / SSF_GYRO; // Store the angular motion for this axis

miscari_unghiulare[ROLL]  = 0.7 * miscari_unghiulare[ROLL]  + 0.3 * gyro_raw[X] / SSF_GYRO;

miscari_unghiulare[PITCH] = 0.7 * miscari_unghiulare[PITCH] + 0.3 * gyro_raw[Y] / SSF_GYRO;

miscari_unghiulare[YAW]   = 0.7 * miscari_unghiulare[YAW]   + 0.3 * gyro_raw[Z] / SSF_GYRO;

Serial.write(int(miscari_unghiulare[YAW]));

}

void calculateGyroAngles() {

gyro_raw[X] -= gyro_offset[X];

gyro_raw[Y] -= gyro_offset[Y];

gyro_raw[Z] -= gyro_offset[Z];

gyro_angle[X] += (gyro_raw[X] / (FREQ * SSF_GYRO));
```

```
gyro_angle[Y] += (-gyro_raw[Y] / (FREQ * SSF_GYRO)); // Change sign to match the accelerometer's one

gyro_angle[Y] += gyro_angle[X] * sin(gyro_raw[Z] * (PI / (FREQ * SSF_GYRO * 180)));

gyro_angle[X] -= gyro_angle[Y] * sin(gyro_raw[Z] * (PI / (FREQ * SSF_GYRO * 180)));

}

void calculateAccelerometerAngles() {

// Calculate total 3D acceleration vector : √(X² + Y² + Z²)

acc_total_vector = sqrt(pow(acc_raw[X], 2) + pow(acc_raw[Y], 2) + pow(acc_raw[Z], 2));

// To prevent asin to produce a NaN, make sure the input value is within [-1;+1]

if (abs(acc_raw[X]) < acc_total_vector) {

acc_angle[X] = asin((float)acc_raw[Y] / acc_total_vector) * (180 / PI); // asin gives angle in radian. Convert to degree multiplying by 180/pi

}

if (abs(acc_raw[Y]) < acc_total_vector) {

acc_angle[Y] = asin((float)acc_raw[X] / acc_total_vector) * (180 / PI);

}

}

void pidController() {

float yaw_pid      = 0;

float pitch_pid    = 0;

float roll_pid     = 0;

int   throttle     = lungime_impuls[mode_mapping[THROTTLE]];

lungime_impuls_esc1 = throttle;
```

```
lungime_impuls_esc2 = throttle;

lungime_impuls_esc3 = throttle;

lungime_impuls_esc4 = throttle;

if (throttle >= 1012) {

// PID = e.Kp + ∫e.Ki + Δe.Kd

yaw_pid   = (errors[YAW]   * Kp[YAW])   + (error_sum[YAW]   * Ki[YAW])   + (delta_err[YAW]
* Kd[YAW]);

pitch_pid = (errors[PITCH] * Kp[PITCH]) + (error_sum[PITCH] * Ki[PITCH]) + (delta_err[PITCH]
* Kd[PITCH]);

roll_pid  = (errors[ROLL]  * Kp[ROLL])  + (error_sum[ROLL]  * Ki[ROLL])  + (delta_err[ROLL]  *
Kd[ROLL]);

yaw_pid   = minMax(yaw_pid, -400, 400);

pitch_pid = minMax(pitch_pid, -400, 400);

roll_pid  = minMax(roll_pid, -400, 400);

lungime_impuls_esc1 = throttle - roll_pid - pitch_pid + yaw_pid;

lungime_impuls_esc2 = throttle + roll_pid - pitch_pid - yaw_pid;

lungime_impuls_esc3 = throttle - roll_pid + pitch_pid - yaw_pid;

lungime_impuls_esc4 = throttle + roll_pid + pitch_pid + yaw_pid;

}

lungime_impuls_esc1 = minMax(lungime_impuls_esc1, 1100, 2000);

lungime_impuls_esc2 = minMax(lungime_impuls_esc2, 1100, 2000);

lungime_impuls_esc3 = minMax(lungime_impuls_esc3, 1100, 2000);

lungime_impuls_esc4 = minMax(lungime_impuls_esc4, 1100, 2000);
```

```
}

void calculateErrors() {

errors[YAW]   = miscari_unghiulare[YAW]   - pid_set_points[YAW];

errors[PITCH] = miscari_unghiulare[PITCH] - pid_set_points[PITCH];

errors[ROLL]  = miscari_unghiulare[ROLL]  - pid_set_points[ROLL];

error_sum[YAW]   += errors[YAW];

error_sum[PITCH] += errors[PITCH];

error_sum[ROLL]  += errors[ROLL];

error_sum[YAW]   = minMax(error_sum[YAW],   -400/Ki[YAW],   400/Ki[YAW]);

error_sum[PITCH] = minMax(error_sum[PITCH], -400/Ki[PITCH], 400/Ki[PITCH]);

error_sum[ROLL]  = minMax(error_sum[ROLL],  -400/Ki[ROLL],  400/Ki[ROLL]);

delta_err[YAW]   = errors[YAW]   - previous_error[YAW];

delta_err[PITCH] = errors[PITCH] - previous_error[PITCH];

delta_err[ROLL]  = errors[ROLL]  - previous_error[ROLL];

previous_error[YAW]   = errors[YAW];

previous_error[PITCH] = errors[PITCH];

previous_error[ROLL]  = errors[ROLL];

}

void configureChannelMapping() {

mode_mapping[YAW]     = CANAL4;

mode_mapping[PITCH]   = CANAL2;

mode_mapping[ROLL]    = CANAL1;
```

```
mode_mapping[THROTTLE] = CANAL3;
}
void setupMpu6050Registers() {
// Configuratie power management
Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
Wire.write(0x6B);                    // Request PWR_MGMT_1 de la registru
Wire.write(0x00);                    // Aplica configuratia la registru
Wire.endTransmission();              // Stop
// Configura gyro sensitivitate
Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
Wire.write(0x1B);                    // Request GYRO_CONFIG de la registru
Wire.write(0x08);                    // Aplica configuratia la registru : ±500°/s
Wire.endTransmission();              // Stop
// Configuratia accelerometrului
Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
Wire.write(0x1C);                    //Request ACCEL_CONFIG de la registru
Wire.write(0x10);                    // Aplica configuratia la registru : ±500°/s
Wire.endTransmission();              //Stop
// Configuratie filtru digital trece jos
Wire.beginTransmission(MPU_ADDRESS); // Incepe comunicatia cu MPU
Wire.write(0x1A);                    // Request CONFIG de la registru
Wire.write(0x03);                    // Seteaza filtru digital trece jos la ~43Hz
```

```
Wire.endTransmission();              // Stop
}
void calibrateMpu6050() {
int max_samples = 2000;
for (int i = 0; i < max_samples; i++) {
readSensor();
gyro_offset[X] += gyro_raw[X];
gyro_offset[Y] += gyro_raw[Y];
gyro_offset[Z] += gyro_raw[Z];
//Genereaza impuls de comandă către ESC-uri pentru a le arma
PORTD |= B11110000;      // Seteaza pin #4 #5 #6 #7 pe HIGH
delayMicroseconds(1000); // Stai 1000µs
PORTD &= B00001111;      // Masca ce pune pinii pe LOW
// asteapta 3 ms pentru urmatorul loop
delay(3);
}
// Calcul compensare medie
gyro_offset[X] /= max_samples;
gyro_offset[Y] /= max_samples;
gyro_offset[Z] /= max_samples;
}
float minMax(float value, float min_value, float max_value) {
```

```
if (value > max_value) {

value = max_value;

} else if (value < min_value) {

value = min_value;

}

return value;

}

bool isRULARE() {

if (status == OPRIT && lungime_impuls[mode_mapping[YAW]] <= 1012 &&
lungime_impuls[mode_mapping[THROTTLE]] <= 1012) {

status = PORNIRE;

}

if (status == PORNIRE && lungime_impuls[mode_mapping[YAW]] == 1500 &&
lungime_impuls[mode_mapping[THROTTLE]] <= 1012) {

status = RULARE;

resetPidController();

resetGyroAngles();

}

if (status == RULARE && lungime_impuls[mode_mapping[YAW]] >= 1988 &&
lungime_impuls[mode_mapping[THROTTLE]] <= 1012) {

status = OPRIT;

stopAll();

}
```

```
return status == RULARE;
}
void resetGyroAngles() {
gyro_angle[X] = acc_angle[X];
gyro_angle[Y] = acc_angle[Y];
}
void stopAll() {
lungime_impuls_esc1 = 1000;
lungime_impuls_esc2 = 1000;
lungime_impuls_esc3 = 1000;
lungime_impuls_esc4 = 1000;
}
void resetPidController() {
errors[YAW]   = 0;
errors[PITCH] = 0;
errors[ROLL]  = 0;
error_sum[YAW]   = 0;
error_sum[PITCH] = 0;
error_sum[ROLL]  = 0;
previous_error[YAW]   = 0;
previous_error[PITCH] = 0;
previous_error[ROLL]  = 0;
```

```
}

void calculateSetPoints() {

pid_set_points[YAW]   = calculateYawSetPoint(lungime_impuls[mode_mapping[YAW]],
lungime_impuls[mode_mapping[THROTTLE]]);

pid_set_points[PITCH] = calculateSetPoint(masuratori[PITCH],
lungime_impuls[mode_mapping[PITCH]]);

pid_set_points[ROLL]  = calculateSetPoint(masuratori[ROLL],
lungime_impuls[mode_mapping[ROLL]]);

}

float calculateSetPoint(float angle, int channel_pulse) {

float level_adjust = angle * 15; // Value 15 limits maximum angle value to ±32.8°

float set_point    = 0;

if (channel_pulse > 1508) {

set_point = channel_pulse - 1508;

} else if (channel_pulse <  1492) {

set_point = channel_pulse - 1492;

}

set_point -= level_adjust;

set_point /= 3;

return set_point;

}

float calculateYawSetPoint(int yaw_pulse, int throttle_pulse) {

float set_point = 0;
```

```
if (throttle_pulse > 1050) {
// There is no notion of angle on this axis as the quadcopter can turn on itself
set_point = calculateSetPoint(0, yaw_pulse);
}
return set_point;
}
void compensateBatteryDrop() {
if (isBatteryConnected()) {
lungime_impuls_esc1 += lungime_impuls_esc1 * ((1240 - battery_voltage) / (float) 3500);
lungime_impuls_esc2 += lungime_impuls_esc2 * ((1240 - battery_voltage) / (float) 3500);
lungime_impuls_esc3 += lungime_impuls_esc3 * ((1240 - battery_voltage) / (float) 3500);
lungime_impuls_esc4 += lungime_impuls_esc4 * ((1240 - battery_voltage) / (float) 3500);
}
}
bool isBatteryConnected() {
battery_voltage = battery_voltage * 0.92 + (analogRead(0) + 65) * 0.09853;
return battery_voltage < 1240 && battery_voltage > 800;
}
ISR(PCINT0_vect) {
timp_prezent = micros();
// Channel 1 -----------------------------------------------
if (PINB & B00000001) {
```

```
if (stare_anterioara[CANAL1] == LOW) {
stare_anterioara[CANAL1] = HIGH;
timer[CANAL1] = timp_prezent;
}
} else if (stare_anterioara[CANAL1] == HIGH) {
stare_anterioara[CANAL1] = LOW;
lungime_impuls[CANAL1] = timp_prezent - timer[CANAL1];
}
// Channel 2 ------------------------------------------------
if (PINB & B00000010) {
if (stare_anterioara[CANAL2] == LOW) {
stare_anterioara[CANAL2] = HIGH;
timer[CANAL2] = timp_prezent;
}
} else if (stare_anterioara[CANAL2] == HIGH) {
stare_anterioara[CANAL2] = LOW;
lungime_impuls[CANAL2] = timp_prezent - timer[CANAL2];
}
// Channel 3 ------------------------------------------------
if (PINB & B00000100) {
if (stare_anterioara[CANAL3] == LOW) {
stare_anterioara[CANAL3] = HIGH;
```

```
timer[CANAL3] = timp_prezent;
}
} else if (stare_anterioara[CANAL3] == HIGH) {
stare_anterioara[CANAL3] = LOW;
lungime_impuls[CANAL3] = timp_prezent - timer[CANAL3];
}

// Channel 4 ------------------------------------------------
if (PINB & B00001000) {
if (stare_anterioara[CANAL4] == LOW) {
stare_anterioara[CANAL4] = HIGH;
timer[CANAL4] = timp_prezent;
}
} else if (stare_anterioara[CANAL4] == HIGH) {
stare_anterioara[CANAL4] = LOW;
lungime_impuls[CANAL4] = timp_prezent - timer[CANAL4];
}
}
void citireSenzorGaz()
{
#if(GAS_SENSOR_PRESENT)
return analogRead(A3);
```

```
#elif

return ;

#endif

}

#if(GEYGER_SENSOR_PRESENT)

ISR(PCINT0_vect) {

// Geiger Click

if (PINB & B00010000) {

ticks = ticks + 1;

}

}

#endif

//endif GEYGER_SENSOR_PRESENT

#if(GEYGER_SENSOR_PRESENT)

ISR(TIMER1_OVF_vect)        // intrerupere generata de timer la interval de o secunda

{

Serial.Write((int)ticks);

ticks = 0;

TCNT1 = timer1_counter;   // reseteaza timer

}

#endif

//endif GEYGER_SENSOR_PRESENT
```

B.CD / DVD

Autorul atașează în această anexă obligatorie, versiunea electronică a aplicației, a acestei lucrări, precum și prezentarea finală a tezei.

